↖ Topic Index

Programming Language Support — Emacs Lisp

Description	Keystroke	Function	Note

Emacs Lisp
○ **Help & Customize**
• Describe function & args
• helpful
○ **eldoc,** eldoc-box
• Load-path control
• Elisp Libraries
• **Extra Modes:**
  • highlight-defined
  • lispy
  • parinfer
  • rainbow-delimiters
  • show-paren-mode
  • toggle lisp modes
  ○ hide/show docstrings
• ELisp Evaluation
• Lisp shell
○ Render markup in comments
• **Writing new code:**
  • Suggest
  • Tempo Skeletons
• Code Completion & Spell Check
○ **Semantic Editing**
  • SemEd Kill
  • SemEd - Parenthesis
  • SemEd - Mark
  • SemEd - Indenting
○ **Navigation**
  • toggle superword-mode
  • to definition (xref)
  • to references
  • to begin/end of forms
  ○ to next/previous form
  ○ across forms
  • by sentences
○ **Macro Expansion**
• **Code analysis**
  • Flycheck
  • relint - regexp lint
  • elisp-depend
  ○ elisp-depmap
• **Compiling**
• ERT regression testing
○ Debugging - Debug
○ Debugging - EDebug
○ Profiler
 	
Last updated on:
	The top level layers of Emacs are written in **Emacs Lisp**. Emacs is a powerful Emacs Lisp based editor and environment.
	Emacs supports the following major modes for Emacs Lisp: **emacs-lisp-mode** (for editing), **lisp-interaction-mode** (for evaluating).
	• Some of the key bindings listed in this table are available from all modes or some other modes (like the PEL key bindings highlighted with light green).
	• Some other are context sensitive and only available for the Emacs Lisp major mode (like the PEL `<f12>` or `M-<f12>` key prefixes, which are highlighted in darker green). Those can also be accessed via the `<f11> SPC l` prefix. *These are not all shown in the following rows to save space.*
	  • Type `<f11> SPC l l` followed by the library name to open an Emacs library file ***from any buffer***. *These key bindings are not always shown in tables.*
	  • Type `<f12> l l` from an emacs-lisp-mode buffer; it's a mode-specific binding. Both support completion (with `TAB`, `M-TAB` or `C-M-i`).
	• Note that some of the commands are meant to be used regardless of the mode, but were documented in this table because they are available everywhere, are essentially controlling or explicitly using the Emacs Lisp engine or environment in such a way so the user must be aware of Emacs Lisp and the available commands.
	• A lot of information is available within Emacs. See **Σ Help/Info** to learn how to access it.
	
	PEL can install and activate several external packages for editing Emacs Lisp file. Activate the corresponding **pel-use-** customizable user-option:		

	lispy (or **this fork**)	**pel-use-lispy** ☝ when changing value, move the old one out of the ~/.emacs.d/elpa directory.	Very useful, powerful, elegant minor mode to edit Lisp languages. See **𝔓𝔩- Lispy** for more info. Set **pel-use-lispy** the following values to install a specific implementation: • **t** : use the original **abo-abo/lispy** (which has not been updated for a while) • **use-enzuru-lispy,** to use the temporary **enzuru fork.**
	parinfer **parinfer-rust-mode**	**pel-use-parinfer**	Supports the **ParInfer** editing mode. Installs the package selected by **pel-use-parinfer** value: • **t**: installs **parinfer** , a fork of the original code. • **use-parinfer-rust-mode**: **parinfer-rust-mode**, which use a back-end written in **Rust**.
	rainbow-delimiters	**pel-use-rainbow-delimiters**	Minor-mode that highlight nested parentheses, brackets, and braces with different colours according to their depth.
	macrostep	**pel-use-macrostep**	Expand Emacs Lisp macro at point. Very useful to debug /understand macros.
	lisp-docstring-toggle	**pel-use-lisp-docstring-toggle**	Toggle visibility of Lisp languages docstrings.
	highlight-defined	**pel-use-highlight-defined**	Minor-mode that highlights defined (loaded) symbols.
	smex	**pel-use-smex**	Enhancement to `M-x` that provides Ido-completion.
	eros-mode	**pel-use-eros**	**E**valuation **R**esult **O**verlay**S** for Emacs Lisp, enhances **eval-last-sexp** by showing result in overlay instead of the echo area.
	suggest.el	**pel-use-suggest**	Suggest mode: discover elisp functions by examples.
	elisp-refs	**pel-use-elisp-refs**	Find references to symbol in code already loaded.
	relint	**pel-use-relint**	Analyze validity of regular expressions in the buffer.
	esup	**pel-use-esup**	Emacs start up profiler. For GUI mode only.
	elisp-depend	**pel-use-elisp-depend**	List library dependencies of an elisp file.
	elisp-depmap	**pel-use-elisp-depmap**	Generate Elisp code dependency **Graphviz** Dot diagrams. For GUI mode only.
	eldoc-box	**pel-use-eldoc-box**	Provide eldoc information inside a child frame. Of limited value. For GUI mode only.
	Emacs Lisp References: 2026-03-17	• Emacs Lisp Manual • An Introduction to Programming in Emacs Lisp - for a gentle (and slow) introduction. • Emacs Lisp Guide, a quick overview of Emacs Lisp and extensions. 💡After reading on Emacs Lisp read on Common Lisp!	
Open this PDF file. See also: **Σ Help/Info**	`<f11> SPC l <f1>` `<f12> <f1>`	(**pel-help-pdf** &optional N)	Open the **𝔛𝔓𝔇 - Emacs Lisp** local PDF (without argument prefix). With positive argument prefix (like `C-u` or `M-2`) prompts for selection of secondary topic PDF: **𝔓𝔩- Lispy** or **𝔛𝔐 - Emacs Lisp Types** PDF instead. With negative numeric argument, opens the corresponding GitHub raw PDF web page. If the **pel-flip-help-pdf-arg** user-option is set the impact of argument sign is flipped: open the GitHub page for positive numeric argument and local PDF for negative.
Σ Customize PEL ELisp support	`<f11> SPC l <f2>` `<f12> <f2>`	(**pel-customize-pel** &optional OTHER-WINDOW)	Customize PEL Elisp support. • If OTHER-WINDOW is non-nil (use `C-u`), display in another window.
Σ Customize Emacs Elisp support	`<f11> SPC l <f3>` `<f12> <f3>`	(**pel-customize-library** &optional OTHER-WINDOW)	Customize Emacs Elisp support: checkdoc, editing-basics, elint, eldoc, eros, highlight-defined, lisp, lispy, lisp-docstring-toggle, rainbow-delimited, suggest. • If OTHER-WINDOW is non-nil (use `C-u`), display in another window.
Getting Code Help See also: **Σ Help/Info** • **𝔓𝔩- Lispy**	Use the following keys to pop information inside the current window (if small enough) or into a help buffer. • The `<f12> 1` and `<f12> 2` PEL keys are available even when lispy mode is off. • See the **Σ Help/Info** table for more commands you can use to get help about Emacs Lisp code and Emacs in general. 📦These require the **lispy** external package. PEL downloads, installs and activates **lispy** when the **pel-use-lispy** user option is set to **t**.		
Describe function at point See also: • **Σ Help/Info** • **𝔓𝔩- Lispy**	`C-1` `<f12> 1`	(**lispy-describe-inline**)	Toggle displaying documentation of ELisp function at point: 'lispy--current-function' inline. • If docstring is small enough it is displayed in a pop-up box above point. Otherwise it is displayed inside a *lispy-help* buffer. • The **<f12> 1** key can be used even when lispy mode is not active (as long as it is installed). It works in terminal mode.
Describe function arguments	`C-2` `<f12> 2`	(**lispy-arglist-inline**)	Toggle displaying of argument list of ELisp function at point. • The **<f12> 2** key can be used even when lispy mode is not active. It works in terminal mode.
Helpful - extended help for Emacs with more contextual information	The helpful external package provides the same help information provided by Emacs with more contextual information and extra links inside a separate buffer that you can close by typing `q` . It also provides links to manual, ability to search in source, etc… 📦This requires the **helpful** external package PEL installs and activates it when the **pel-use-helpful** user-option is set. ☝These commands provide a lot more information than standard Emacs help. Use then to debug, trace, look at references, search where they are used, etc…		
Help for function/macro/special form	`<f1> <f2> a`	(**helpful-callable** SYMBOL)	Show help for function, macro or special form named SYMBOL. • Prompts for symbol, defaulting to the symbol at point. Supports tab-completion.
Help for command	`<f1> <f2> c`	(**helpful-command** SYMBOL)	Show help for interactive function named SYMBOL. • Prompts for symbol, defaulting to the symbol at point. Supports tab-completion.
Help for function	`<f1> <f2> f`	(**helpful-function** SYMBOL)	Show help for function named SYMBOL. • Prompts for symbol, defaulting to the symbol at point. Supports tab-completion.
Help for key	`<f1> <f2> k`	(**helpful-key** KEY-SEQUENCE)	Show help for interactive command bound to KEY-SEQUENCE. • Prompts for key sequence.
Help for macro	`<f1> <f2> m`	(**helpful-macro** SYMBOL)	Show help for macro named SYMBOL. • Prompts for symbol, defaulting to the symbol at point. Supports tab-completion.
Help for symbol	`<f1> <f2> o`	(**helpful-symbol** SYMBOL)	Show help for SYMBOL, a variable, function or macro. • Prompts for symbol, defaulting to the symbol at point. Supports tab-completion.
Help for variable	`<f1> <f2> v`	(**helpful-variable** SYMBOL)	Show help for variable named SYMBOL. • Prompts for symbol, defaulting to the symbol at point. Supports tab-completion.
Help for symbol at point	`<f1> <f2> .`	(**helpful-at-point**)	Show help for the symbol at point.

Description	Keystroke	Function	Note
Σ Eldoc Documentation	Display code documentation based on **Emacs-lisp docstrings**. eldoc is active for all lisp-based modes.		
Show eldoc setup information for the current buffer	`<f11> SPC * d ?` `<f12> <f4> d ?`	(**pel-eldoc-setup-info** &optional APPEND)	Display eldoc setup information inside a *pel-eldoc-info* buffer with buttons providing quick access to the customization buffer of each variable shown. `C-u` prefix: append info in buffer.
Toggle eldoc-mode Emacs Lisp Documentation Lookup	`<f12> <f4> d d`	(**eldoc-mode** &optional ARG)	Toggle echo area display of Lisp objects at point (ElDoc buffer local minor mode). • When enabled, display info in the echo area about current variable or function.
	If point on a documented variable, display the first line of that variable's doc string., otherwise display the function argument list.		
Eldoc-box	📦 Require **eldoc-box** external package. ☑ activated by **pel-use-eldoc-box** user option. Show eldoc info in a box instead of echo area. For GUI mode only.		
Toggle eldoc-box at point	`<f12> <f4> d b`	(**eldoc-box-hover-at-point-mode** &optional ARG)	Toggle eldoc-box that displays eldoc text at point. • You can use `C-g` to hide the doc.
Toggle eldoc-box on upper corner	`<f12> <f4> d B`	(**eldoc-box-hover-mode** &optional ARG)	Displays hover documentations in a childframe. • The default position of childframe is upper corner.
Load-Path Control See **also: Σ Help/Info**	Emacs can evaluate Emacs Lisp forms that it knows about: forms in files already loaded or whose names are associated with a file to autoload. Emacs finds files to load in the **load-path** variable. Add a directory to **load-path** with the following command and explicitly load a file with the next command. See **Σ Help/Info** for commands to show the value of the load-path, statistics, and list shadowed files.		
Add a directory to load-path ☝ Useful for testing elisp code.	• `<f12> <f4> l d`	(**pel-add-dir-to-loadpath** DIR)	Add a directory to Emacs variable 'load-path' if not already in the list. Interactively display the # of directories in the list and whether the operation succeeded or not. • Use it when working in files path of packages that are not in your standard Emacs load-path.
Load Emacs Lisp file	• `<f12> l f` `<f11> SPC l l f`	(**load-file** FILE)	Load the Emacs Lisp file named FILE. • Emacs prompts for the .el or .el.gz file name.
Load current Emacs List file	• `<f12> l v` `<f11> SPC l l v`	(**pel-load-visited-file** &optional USE-ELC)	Load the Emacs Lisp file visited in the current buffer. • By default load the source code file (the .el file). • With any prefix argument, load the byte-compiled file instead.
Elisp Libraries	The commands below are used to find and load Emacs Lisp **libraries**		
Load a Lisp library from load-path	• `<f12> l L` • `M-<f12> l L` `<f11> SPC l l L`	(**load-library** LIBRARY)	Load the Emacs Lisp library named LIBRARY. • Emacs prompts for LIBRARY, a string, identifying the Emacs Lisp file: no need for the path or the extension, the file is searched searched for in '*load-path*', both with and without '*load-suffixes*' (as well as '*load-file-rep-suffixes*').
Find and open Library file	• `<f12> l l` `<f11> SPC l l l`	(**find-library** LIBRARY)	Find the Emacs Lisp source of LIBRARY. • Interactively, prompt for LIBRARY using the one at or near point.
Locate a library	• `<f12> l c` • `M-<f12> l c` `<f11> SPC l l c`	(**locate-library** LIBRARY &optional NOSUFFIX PATH INTERACTIVE-CALL)	Show the precise file name of Emacs library LIBRARY. • LIBRARY should be a relative file name of the library, a string. • Can omit the suffix (file-name extension) if NOSUFFIX is nil (which is the default, see below). • Searches directories in '*load-path*' like '`<f11> SPC l l L`' to find the file that '`<f11> SPC l l L RET LIBRARY RET`' would load. Optional 2nd arg NOSUFFIX non-nil means don't add suffixes 'load-suffixes' to the specified name LIBRARY.
List available Emacs Lisp packages	• `<f12> l p` • `M-<f12> l p` `<f11> SPC l l p`	(**package-list-packages** &optional NO-FETCH)	Display a list of packages. This first fetches the updated list of packages before displaying, unless a prefix argument NO-FETCH is specified. • The list is displayed in a buffer named '*Packages*', and includes the package's version, availability status, and a short description.
Extra Modes	The following commands can be used to activate or toggle useful modes for Emacs Lisp editing, specially for helping dealing with parenthesis: • **show-paren-mode**, which highlights the parens that matches the one before or after point. • **ParInfer** mode (with either ParInfer Indent Mode or Parinfer Paren Mode) where the parenthesis or indentation is automatically inferred from the other. • **rainbow delimiters** mode, where matching nested parens are highlighted with the same colour. 🎛 To activate them automatically, put their name in the **pel-elisp-activates-minor-modes** user-option. Use **<f12> <f2>** to open customization buffer.		
Toggle semantic parser mode on/off	• `<f12> M-s` • `M-<f12> M-s` `<f11> SPC l M-s`	(**semantic-mode** &optional ARG)	Toggle parser features (Semantic mode). • With a prefix argument ARG, enable Semantic mode if ARG is positive, and disable it otherwise. If called from Lisp, enable Semantic mode if ARG is omitted or nil. • In Semantic mode, Emacs parses the buffers you visit for their semantic content.
Toggle Lispy mode ★★ See also: **𝔓𝔏- Lispy**	`<f12> M-L` `<f11> SPC L M-L`	(**pel-lispy-mode** &optional ARG)	Toggle lispy-mode on/off. Lispy is a minor mode for navigating and editing LISP dialects. • **pel-lispy-mode** calls **lispy-mode**, if enabling: disables **overwrite-mode,** prepares **hydra**. 📦 Requires **lispy** external package. ☑ PEL downloads, installs and configure it when **pel-use-lispy** user option is set to **t**.
Toggle ParInfer mode on/off	• `<f12> M-i` • `M-<f12> M-i` `<f11> SPC l M-i`	(**parinfer-mode** &optional ARG)	Toggle use of the **ParInfer** mode, a mode similar to Lispy. In this mode parenthesis depth or indentation is automatically inferred. ⚠ Current implementation of ParInfer does not support hard tabs for indentation. It untabifies and replace them by spaces. 📦 Requires one of the parinfer packages. ☑ PEL activates via **pel-use-parinfer** user option.
Toggle between ParInfer Indent Mode and Paren Mode 1. 💡 Parinfer Paren Mode can be used to fix incorrectly indented code before using Parinfer Indent Mode.	• `<f12> M-I` • `M-<f12> M-I` `<f11> SPC l M-I`	(**parinfer-toggle-mode**)	Switch ParInfer mode between Indent Mode and Paren Mode. 📦 Requires parinfer external package. ☑ PEL activates it when **pel-use-parinfer** is set to **t**.
	⚠ Note that if the ParInfer mode is not active yet, and it enters ParInfer Indent Mode, the function checks the style of the current buffer and proceed with changing the format after prompting when it finds code that does not conform to the promoted style. The 2 ParInfer modes are: 1. ParInfer **Indent Mode**: • Gives full control of indentation, while ParInfer corrects parens. • Disables the rainbow-delimiter-mode if used, to show closing parens in light gray since they can change as code indentation is changed. • ⚠ When changing to Indent Mode, ParInfer may correct the parentheses format if the code does not corresponds to the promoted style. 2. ParInfer **Paren Mode**: • Gives full control of parens, while ParInfer controls indentation. • Activates rainbow-delimiters-mode if available, showing matching parens in same colors.		
Toggle between Lisp modes	• `<f12> M-l` • `M-<f12> M-l` `<f11> SPC l M-l`	(**pel-toggle-lisp-modes**)	Toggle buffer's LISP mode: 'lisp-interaction-mode' <-> 'emacs-lisp-mode'. ☝ Useful if you want to use `C-j` to evaluate and print value of the sexp before point while editing an Emacs Lisp (.el) file: when editing .el file, Emacs is normally in emacs-lisp-mode where `C-j` is mapped to electric-newline-and-maybe-indent. Temporarily changing to lisp-interaction-mode maps `C-j` to eval-print-last-sexp.
Toggle show-paren mode on/off See also: **Σ Highlight**	`<f12> h (` `<f11> h (`	(**show-paren-mode** &optional ARG)	Toggle visualization of matching parens (Show Paren mode). • With prefix argument ARG, enable Show Paren mode if ARG is positive, and disable it otherwise. • Show Paren mode is a global minor mode. When enabled, any matching parenthesis is highlighted in 'show-paren-style' after 'show-paren-delay' seconds of Emacs idle time.
Enable/Disable coloured highlight of nested blocks (), {},[] See also: **Σ Highlight**	`<f12> h )` `<f11> h )`	(**rainbow-delimiters-mode** &optional ARG)	Highlight nested parentheses, brackets, braces with different colours according to their depth. • Customize the depth and colours via `<f12> <f3>` and select rainbow-delimiters. 📦 Requires: **rainbow-delimiters** ☑ PEL activates when **pel-use-rainbow-delimiters** is **t**.
Toggle Lisp Defined Symbol Highlight	`<f12> h d` `<f11> SPC l h d`	(**highlight-defined-mode** &optional ARG)	Minor mode for highlighting known Emacs Lisp functions and variables. • Customize the depth and colours via `<f12> <f3>` and select highlight-defined. 📦 Requires: **highlight-defined** ☑ PEL activates when **pel-use-highlight-defined** is set to **t**.

Description	Keystroke	Function	Note
Hide/Show docstrings	Toggle visibility of Lisp languages docstrings. Requires 📦 **lisp-docstring-toggle** activated by **pel-use-lisp-docstring-toggle**		
Toggle lisp docstring toggle mode	`<f12> ' m`	(**lisp-docstring-toggle-mode** &optional *ARG*)	Toggle the minor mode for toggling Lisp docstring visibility. • This mode provides keybindings for docstring toggling commands: • `C-c C-d t` - Toggle all docstrings in buffer • `C-c C-d .` - Toggle docstring at point • `C-c C-d D` - Show debug information
Toggle visibility of all docstrings in buffer	`<f12> ' '`	(**lisp-docstring-toggle**)	Toggle the visibility of all docstrings in the buffer. • When docstrings are visible, hide them. When hidden, show them. • The hiding style is controlled by '**lisp-docstring-toggle-hide-style**': • 'complete': Hide entire docstring content • 'partial': Show first N characters (see 'lisp-docstring-toggle-partial-chars') • 'first-line': Show only the first line
Toggle visibility of docstring in the form at point	`<f12> ' .`	(**lisp-docstring-toggle-at-point**)	Toggle visibility of the docstring in the current form at point. If the docstring is currently hidden (has an overlay), remove the overlay to show it. If visible, create an overlay to hide it.
Show all detected docstrings inside a buffer with navigation links.	`<f12> ' d`	(**lisp-docstring-toggle-debug-show-docstring-snippets**)	Open a buffer showing all detected docstrings with navigation links. • This command is useful for: • Verifying that docstrings are detected correctly • Navigating to specific docstrings in the source buffer • Debugging detection issues The debug buffer shows for each docstring: • Clickable position links (BEG and END) • Character count • First 2 lines of content • Last 2 lines of content Clicking a position number jumps to that location in the source buffer. If no docstrings are found, display a message instead of opening a buffer.
Emacs Lisp Evaluation. See Evaluating Elisp in Emacs	GNU Emacs is implemented in Emacs Lisp with low level code written in C. Some of the functions can be used interactively; these functions are called commands. Some of these commands are bound to a key or a combination of keys (called key bindings). ‣ This section shows the commands (and their key bindings) you can use to explicitly evaluate Emacs Lisp code. ‣ The bindings shown in light blue coloured boxes are available in the emacs-lisp-mode and lisp-interaction-mode (the *scratch* buffer) except were noted.		
Execute Emacs Command See also: **Completion/Input**, specially: • `<f11> M-c <f4>` • `<f11> M-c ?` ☝ For Emacs >= 28, when **suggest-key-bindings** is **t**, Emacs completion in most completion modes show key bindings along the command name. **Emacs >= 28**	`<f11> M-x`	(**execute-extended-command** PREFIXARG &optional COMMAND-NAME TYPED)	Read a command name, then read the arguments and call the command. • To pass a prefix argument to the command you are invoking, use a prefix argument. • The `<f11> M-x` key binding is only available when the **smex** external package is activated by PEL **pel-use-smex** user option set to **t**.
	`M-x <command>`	(**smex**)	Same as execute-extended-command but with Ido-based completion. 📦 Requires **smex** external package PEL activates it when **pel-use-smex** user option is **t**.
		• From the prompt you can press **<tab>** to perform completion and to list the names of the Emacs commands available. • To see the list of available commands, type `M-x <tab> <tab>` then press `<tab>` again to scroll the (large) list. • To quit the expansion of this command, type `C-g` or `<Esc> <Esc><Esc>`.	
	`M-X <command>`	(**smex-major-mode-commands**)	Same as execute-extended-command but with Ido-based completion, and limited to commands that are limited to the current major mode. • When smex is not available this key sequence does the same as M-x. 📦 Requires the **smex** external package PEL activates it when **pel-use-smex** user option is **t**.
	`<f11> M-X <command>` ~~`M-X`~~	(**execute-extended-command-for-buffer** PREFIXARG &optional COMMAND-NAME TYPED)	Query user for a command relevant for the current mode, and then execute it. • This is like 'execute-extended-command', but it limits the completions to commands that are particularly relevant to the current buffer. This includes commands that have been marked as being specially designed for the current major mode (and enabled minor modes), as well as commands bound in the active local key maps.
Read & eval mini buffer	`M-:`	(**eval-expression** EXP &optional INSERT-VALUE NO-TRUNCATE CHAR-PRINT-LIMIT)	Read a single Emacs Lisp expression in the mini buffer, evaluate it, and print the value in the echo area.
Toggle eros mode — Evaluation Result OverlayS	• `<f12> E` • `M-<f12> E` `<f11> SPC l E`	(**eros-mode** &optional ARG)	Toggle the eros-mode: where it display Emacs Lisp evaluation results overlays instead of inside the minibuffer. This affects how the next 2 commands display results. 📦 Requires **eros-mode** external package PEL installs the eros-mode when **pel-use-eros** user-option is set to **t**.
Eval sexp before cursor	`C-x C-e`	(**eval-last-sexp** EVAL-LAST-SEXP-ARG-INTERNAL)	Evaluate sexp before point; print value in the echo area. • Interactively, with a non '-' prefix argument, print output into current buffer: ie: `C-u C-x C-e` prints output to the current buffer.
			• 📦 If **eros-mode** is active, print result on temporary buffer overlay instead. With PEL, with **pel-use-eros** on, toggle eros-mode with `<f12> E`.
Eval sexp before cursor and copy result in the kill ring	`<f11> SPC l e e` `<f12> e e`	(**pel-eval-last-sexp-and-copy**)	Evaluate sexp before point; print value in echo area and copy to kill-ring. • Same as above but copies the result inside the kill ring. Yank it anywhere with `C-y`
Evaluate Lisp-Expression (defun) at point • Not restricted to a defun, it supports all definition forms.	`C-M-x`	(**eval-defun** EDEBUG-IT)	Evaluate the top-level form containing point, or after point. • With a prefix argument (`C-u`), instrument the code for Edebug (see edebug section below).
	⚠️ For Emacs < 28: If current defun is actually a call to defvar or defcustom, *evaluating it this way resets the variable using its initial value expression* (using the defcustom's :set function if there is one), **even if the variable already has some other value**. (*Normally defvar and defcustom do not alter the value if there already is one.*) In an analogous way, evaluating a defface overrides any customizations of the face, so that it becomes defined exactly as the defface expression says. **Fixed in Emacs 28.**		
Evaluate Lisp S-expression before point	`C-j`	(**eval-print-last-sexp** &optional EVAL-LAST-SEXP-ARG-INTERNAL)	Evaluate sexp before point; print value into current buffer. • For example, use this in the *Scratch* buffer: place the cursor after an expression and type `C-j` to evaluate the expression. Emacs evaluate, run the expression & prints the returned value.
	⚠️ This `C-j` binding is **only available in the Lisp-Interaction mode** (the default mode of the *Scratch* buffer but not the default mode for editing Emacs Lisp files. You can use `<f12> m L`, (**pel-toggle-lisp-modes**), to temporarily change mode and activate the binding in the .el file buffer.		
Insert a new line	`C-j`	(**electric-newline-and-maybe-indent**)	Insert a newline. • This binding is in effect in the emacs-lisp-mode.
Eval all Emacs Lisp expressions in the buffer	• `<f12> e b` • `M-<f12> e b` `<f11> SPC l e b`	(**eval-buffer** &optional BUFFER PRINTFLAG FILENAME UNIBYTE DO-ALLOW-PRINT)	Execute the accessible portion of current buffer as Lisp code. • You can use `C-x n n` (narrowing) to limit the part of buffer to be evaluated. • This function preserves the position of point.
Evaluate all Emacs Lisp expressions in region	• `<f12> e r` • `M-<f12> e r` `<f11> SPC l e r`	(**eval-region** START END &optional PRINTFLAG READ-FUNCTION)	Execute the region as Lisp code. • This function preserves the position of point.
ELisp Shell	Use the Interactive Emacs Lisp Mode (ielm) shell to test various Emacs Lisp forms.		
Emacs Lisp shell See also: **Shells**	• `<f12> z`	(**ielm**)	Open the Interactive Emacs Lisp Mode buffer where you can interactively evaluate Emacs Lisp expressions, a REPL for Emacs Lisp. Mode:= inferior-emacs-lisp-mode. • Switches to the buffer '***ielm***', or creates it if it does not exist.
	• `<f11> z l` • `<f11> SPC l z`		
Evaluate current line in ielm	`C-j`	(**ielm-send-input** &optional FOR-EFFECT)	Evaluate the Emacs Lisp expression after the prompt.

Description	Keystroke	Function	Note
Render markup in comments	The following commands are used to create images from specific markup code embedded inside Emacs Lisp source code comments. This can be useful when using these markup languages to describe UML diagrams or finite-state machines for example.		
Preview UML diagram from plantUML source in current plantUML region of commented source code See also: **M PlantUML**	• `<f12> u` • `M-<f12> u` `<f11> SPC l u`	(**pel-render-commented-plantuml** PREFIX &optional POS)	Render the PlantUML markup embedded in current mode comment. 📦 Requires **plantuml-mode** external package, activated by **pel-use-plantuml** user option. • Use region if identified otherwise use PlantUML block at point. • Uses prefix (as PREFIX) to choose where to display it: 4 (with **C-u**) -> new window, 16 (with **C-u C-u**) -> new frame else -> new buffer • This can be used inside buffer using **any** major mode, when PlantUML markup is embedded inside source code comment. ☝ Use this in source code to describe your code architecture with PlantUML markup, then generate the UML rendering by moving point inside the PlantUML block and issuing this command.

Writing New Code	The tools listed in this section provide help for writing Emacs Lisp code. • The lispy-mode (see **Pix- Lispy**) is a very powerful mode that helps navigating and editing emacs-lisp code.		
Suggest	Learn new Emacs Lisp functions by getting suggestions from input data and requested output data: **example-driven development**!		
Open suggest buffer	`<f12> S` `<f11> SPC l S`	(**suggest**)	Open a Suggest buffer that provides suggestions for the inputs and outputs given. 📦 **Requires suggest.el**. PEL activates when **pel-use-suggest** user-option is **t**.
Tempo skeletons for Emacs Lisp See also: • **Inserting Text** for more info and information about tempo skeleton and yasnippet template-based text insertion).	PEL provides support for flexible text template insertion through the Emacs built-in **tempo skeleton** mechanism. • PEL creates key bindings to invoke the skeletons in the supported major modes, using the same key prefix sequence for each mode: `<f12> <f12>`, with the same key bindings for equivalent concepts (such as file header block) as much as possible. • 🎛 Several aspects of the PEL Emacs Lisp Source Code Style is controlled by the user options inside the **pel-elisp-code-style** group. This group can be edited with **<f12> <f2>** from an emacs-lisp mode buffer and include the following options: • **pel-elisp-skel-insert-file-timestamp** : set whether an automatically updated timestamp is inserted in the file header block. • **pel-elisp-skel-use-separators** : set whether blocks use horizontal separator lines. • **pel-elisp-skel-package-name** : set whether the package name is shown. • **pel-elisp-skel-with-license** : set whether file header blocks use open source software license text controlled by lice. ☝ Emacs user options by default take effect globally. But by using file and directory variables (see **File/Directory Variables**) they can also be used to take effect on a single file or all files inside a directory tree. So by default, the user options that control the PEL tempo template take effect globally. If you want to change the behaviour for only one file, write the user option control block at the end of that file. If you want to control the behaviour of the PEL tempo templates for all files inside a directory tree create a .dir-locals file and store the values of the relevant options variables inside that file. This allows you to control the user options affecting the format of the tempo templates precisely and does not affect what you actually type. • Once a skeleton was just entered (or later by activating the pel-tempo-mode) you can move to the next or previous point of interest (so called *tempo-marks)* with the standard tempo-mode keys `C-c M-f` and `C-c M-b` or some other keys like `C-c .` and `C-c ,`.		
Customize PEL Emacs Lisp Skeletons layout	`<f12> <f12> <f2>`	(**pel-customize-pel** &optional OTHER-WINDOW)	Customize PEL Emacs Lisp skeleton layout. • If OTHER-WINDOW is non-nil (use **C-u**), display in another window.
Insert a file header	`<f12> <f12> h`	(**pel-elisp-file-header**)	Insert a large header includes all normal header fields plus separators. • Prompts for file purpose and insert a complete file header block with the file name, its purpose, setting lexical-binding, automatically updated timestamp if required by customization, package name, license text if required by customization, commentary, dependencies and code sections possibly separated by block separators if required by customization and the file ending code. • Automatically activates the PEL tempo skeleton mode so you can move to the target points where extra text must be entered to complete the template.
Toggle pel-tempo-mode	`<f12> <f12> SPC`	(**pel-tempo-mode** &optional ARG)	Toggle PEL tempo mode on/off. When active mode-line shows pel-tempo-mode lighter: ‡ PEL tempo mode activates `C-c .` and `C-c ,` as well as to `C-c C-.` and `C-c C-,` key bindings to navigate across tempo mark hot-spots. The second set are only available when Emacs runs in graphics mode. ☝ When a skeleton is inserted via the execution of one of the pel-rst-… commands, the pel-tempo-mode is automatically activated.
Jump to next tempo mark	• `C-c M-f` • `C-c .` • `C-c C-.`	(**tempo-forward-mark**)	Jump to the next mark in 'tempo-back-mark-list': the location where code must be updated inside the inserted skeleton. • These key key bindings are only available when pel-tempo-mode is active.
Jump to previous tempo mark	• `C-c M-b` • `C-c ,` • `C-c C-,`	(**tempo-backward-mark**)	Jump to the previous mark in 'tempo-back-mark-list': the location where code must be updated inside the inserted skeleton. • These key binding are only available when pel-tempo-mode is active.
Tempo Template Tag Insertion	`<f12> <f12> <f12>`	(**tempo-complete-tag** &optional SILENT)	Look for a tag and expand it. ☝ Instead of using the `<f12> <f12>` key bindings above, you can type the template name (shown in the title column like "if", "case", etc) completely or partially and then hit `<f12> <f12> <f12>`. A completion buffer opens up if the template name is incomplete (or empty in which case the buffer lists **all** available template names). Select the template name and hit RET. Emacs expands the template. • All the tags in the tag lists in 'tempo-local-tags' (this includes 'tempo-tags') are searched for a match for the text before the point. The way the string to match for is determined can be altered with the variable 'tempo-match-finder'. If 'tempo-match-finder' returns nil, results are the same as no match at all. • If a single match is found, the corresponding template is expanded in place of the matching string. • If a partial completion or no match at all is found, and SILENT is non-nil, the function will give a signal. • If a partial completion is found and 'tempo-show-completion-buffer' is non-nil, a buffer containing possible completions is displayed. ☛ Since only one template is available in emac-lisp-mode, the usefulness of this command is limited here.
Code Completion & Spell Checking	Code auto completion and spell checking is available for Emacs Lisp source code files. Spell checking should be restricted to comments and strings, and code completion available everywhere else.		
Complete a partially typed word or Emacs Lisp symbol See also: • **Auto-Completion** • **Spell Checking**	• `M-<tab>` • `C-M-i` • `<Esc> <tab>` • `C-.`	(**completion-at-point**)	Perform completion on the text around point. The completion method is determined by '**completion-at-point-functions**'. For Emacs Lisp code this is normally (**tags-completion-at-point-function**) which uses the tag facility to identify the choices, shown in a completion buffer. ☝ Interaction with Flyspell: • The key binding is affected by Flyspell: when Flyspell mode is active (either for the entire file or just for comment and strings) then the key chord is bound to (**flyspell-auto-correct-word**) instead. However, when the command is issued inside code, then Flyspell invokes code completion function (completion-at-point) such that the completion of the code is done the way it would be normally. • You can use `<f11> $ F` (flyspell-mode &optional ARG) to activate Flyspell or `<f11> $ p` (flyspell-prog-mode) to activate Flyspell but restrict it to spell check comment and strings. See the **Spell Checking** table for more information.
Enter/Leave Flyspell mode See also: **Spell Checking**	`<f11> $ F`	(**flyspell-mode** &optional ARG)	Toggles the use of Flyspell mode. • Mode line shows "Fly" when Flyspell mode is active. • Flyspell mode works like word processors; misspelled words are highlighted. • Use Flyspell Prog mode for code; Flyspell processes all text. • With a prefix argument ARG, enable Flyspell mode if ARG is positive, and disable it otherwise. • Flyspell mode is a buffer-local minor mode. When enabled, it spawns a single ispell/aspell process and checks each word. The default flyspell behavior is to highlight incorrect words. ☞ You should normally not activate Flyspell everywhere in an Emacs Lisp file. However, if you activate it only for comments and strings with `<f11> $ p`, and then if you want to disable it you will have to disable the Flyspell mode completely with `<f11> $ F`.
Enter Flyspell Prog mode See also: **Spell Checking**	`<f11> $ p`	(**flyspell-prog-mode**)	Turn on Flyspell prog mode: turn on Flyspell but restricts it to comments and strings, do not spell check source code itself. Highlight misspellings only in comments or strings. ☞ Note that the command always enables the flyspell-prog-mode, it does not toggle it. If you want to turn spell checking off, you must use the flyspell-mode command. To re-enable Flyspell Prog mode you then flyspell-prog-mode again. ☝ If a hook activates Flyspell Prog mode, you won't need this command. 🎛 PEL provides 2 user options to identify which modes should automatically activate flyspell-mode and flyspell-prog-mode: **pel-modes-activating-flyspell-mode** and **pel-modes-activating-flyspell-prog-mode**.

Description	Keystroke	Function	Note
Semantic Editing	Several of the commands for editing Common Lisp code are also available for other modes and are described in the tables describing the generic Emacs commands (the pages with a title that begin with the character 'Σ'). These commands are repeated here for convenience; their keystroke cell is filled with a pale yellow colour. Several of them are described, with code examples, in the Common Lisp Cookbook - Using Emacs as a Lisp IDE page: this also mostly applies to Emacs Lisp code.		
SemEd - Kill			
Kill next Lisp S-expression See also: • **Σ Cut & Paste**	• **C-M-k** • **<f11> -]**	(**kill-sexp** &optional ARG)	• No argument: kill the next sexp (or the current from the point forward). • With negative sign: kill the previous sexp (the sexp backward). • For example: **M- - C-M-k** kills the sexp backward. • With numeric argument: kill that many sexp in the direction identified by the sign of the argument.
Kill previous Lisp S-expression See also: • **Σ Cut & Paste**	• **C-M-⌫** • **<f11> - [**	(**backward-kill-sexp** &optional ARG)	Kill the sexp (balanced expression) preceding point. • With ARG, kill that many sexps before point. • Negative arg -N means kill N sexps after point. • This command assumes point is not in a string or comment. ⚠️ Note: In some text (like The Common Lisp Cookbook - Using Emacs as a Lisp IDE) , the **C-M-<backspace>** keystroke is being described to kill the previous sexp. This key does not seem to be used anymore. This key chord is normally not accessible in terminal mode as it would map to **C-M-h** instead. The **C-M-⌫** binding only works in terminal mode. Since this key-chord is not the best match for the operation, use **M- - C-M-k** instead or use the PEL **<f11> - [**
Kill Lisp S-Expression at point See also: **Σ Cut & Paste**	**<f11> - x**	(**pel-kill-sexp-at-point**)	Kill the S-Expression at point. The point must be at the opening parenthesis or just after the closing parenthesis.
SemEd - Parentheses	The commands below are used to help dealing with the parentheses (along with the semantic editing navigation commands listed above). Note that when the ParInfer mode is used, these are not required: in that mode you can type the parentheses characters and that will perform the same.		
Insert Parentheses (See also: 𝔓𝔩 Common Lisp, CLCB s4.lisp)	**M- (**	(**insert-parentheses** &optional ARG)	Enclose following ARG sexps in parentheses. • Leave point after open-paren. • A negative ARG encloses the preceding ARG sexps instead. • No argument is equivalent to zero: just insert '()' and leave point between. • If 'parens-require-spaces' is non-nil, this command also inserts a space before and after, depending on the surrounding characters. For Lisp it's best to have this set to non-nil. • If region is active, insert enclosing characters at region boundaries. • This command assumes point is not in a string or comment.
Move past close ')' and reindent (See also: 𝔓𝔩 Common Lisp)	**M-)**	(**move-past-close-and-reindent**)	Move past next ')', delete indentation before it, then indent after it. • Used to add another entry in the parent list.
SemEd - Mark			

Mark region by semantic unit, increase marked region on each invocation.
 	
★Powerful command ★
 	
See also: **Σ Marking**
	• **M-=**
	• **<f11> . =**
		(**er/expand-region** ARG)	Increase selected region by semantic units.
			• With prefix argument expands the region that many times.
			• If prefix argument is negative calls 'er/contract-region'.
			• If prefix argument is 0 it resets point and mark to their state before calling 'er/expand-region' for the first time.
			This command is very powerful: the first time it's typed it selects a word, if you type it again it will expand the selection, and again, and again. The expansions follow the semantics of the current major mode: it is aware of the semantics of several programming languages.
			☛ Once M-= is typed, you can quickly type the following single keys in sequence:
			  • **=** to expand the region,
			  • **-** to contract the region,
			  • **0** to reset the operation.
			If you wait too long, then you have to use **M-=** again to continue the expansion, otherwise the region is de-activated.
			Note that you can also use the following key chords to control the contraction of the selected text without having to worry about time:
			  • **M- M-=** to contract the region
			  • **M-0 M-=** to reset the operation.
			• You can use the cursor keys to expand or contract the region and **C-x C-x** to exchange mark and point to expand the other side of the region.
			📦 This requires the **expand-region** external package. 🤖 Under PEL, activated with ***pel-use-expand-region*** user option.
			• The PEL package uses this command and key binding for it, a popular binding for this command is **C-=** but that key does not work in text terminal mode.
			✂ The standard Emacs binding for **M-=** is normally **count-words-region** used for counting words in region, but PEL provides **<f11> C R** for that.

mark function See also: **Σ Marking**	**C-M-h**	(**mark-defun** &optional ALLOW-EXTEND)	Put mark at end of this defun, point at beginning. • The defun marked is the one that contains point or follows point. • With positive ARG, mark this and that many next defuns; with negative ARG, change the direction of marking. • If the mark is active, it marks the next or previous defun(s) after the one(s) already marked.
mark sexp and balanced expressions See also: **Σ Marking**	• **Esc C-@** • **C-M-@** • **C-M-SPC** • **<f11> . x**	(**mark-sexp** &optional ARG ALLOW-EXTEND)	Set mark ARG sexps (and balanced expressions) from point. • The place mark goes is the same place C-M-f would move to with the same argument. • Interactively, if this command is repeated or (in Transient Mark mode) if the mark is active, it marks the next ARG sexps after the ones already marked. • This command assumes point is not in a string or comment.
SemEd - Indenting	The indentation rules of Common Lisp code differ from the ones for Emacs Lisp. The indentation is controlled by a function bound to the Emacs variable ***lisp-indent-function*** . For Emacs Lisp the function to use is ***lisp-indent-function*** .		
Indent current line (or region)	**<tab>**	(**indent-for-tab-command** &optional ARG)	Indent the current line or region, or insert a tab, as appropriate. • This function either inserts a tab, or indents the current line, or performs symbol completion, depending on 'tab-always-indent'. The function called to actually indent the line or insert a tab is given by the variable 'indent-line-function'. • If a prefix argument is given, after this function indents the current line or inserts a tab, it also rigidly indents the entire balanced expression which starts at the beginning of the current line, to reflect the current line's indentation. • In most major modes, if point was in the current line's indentation, it is moved to the first non-whitespace character after indenting; otherwise it stays at the same position relative to the text. • If 'transient-mark-mode' is turned on and the region is active, this function instead calls 'indent-region'. In this case, any prefix argument is ignored.
Indent lines of list after point See also: **Σ Indentation**	**C-M-q**	(**indent-pp-sexp** &optional ARG)	Indent each line of the list starting just after point, or pretty-print it. • A prefix argument (**C-u**) specifies pretty-printing. Pretty-printing essentially uses more lines as it places the beginning of each list on a new line.
Untabify and re-indent complete buffer with ParInfer	• **<f12> i** • **M-<f12> i** • **<f11> SPC l i**	(**parinfer-auto-fix**)	Untabify whole buffer then reindent whole buffer. 📦 Requires parinfer external package. 🤖 PEL activates it when **pel-use-parinfer** is set to **t**.

Description	Keystroke	Function	Note
Navigation in Elisp • See also: **Navigation**	This current list below describe the specialized commands only. See the others inside **Navigation** You can also use the lispy mode for extra single key commands for navigation across Lisp source code. See **Pl- Lispy**		
	In emacs-lisp-mode, the **superword-mode** is useful since the code uses lisp-case (also called **kebab-case**). Using **superword-mode** helps navigate over complete words. PEL activates **superword-mode** by default in Emacs Lisp mode. To change this use `<f11> t <f2>` to access the customize buffer.		
Toggle superword-mode See also: • **Text Modes** • **Search/Replace**	• `<f12> M-p` • `M-<f12> M-p` `<f11> t m p`	(**superword-mode** &optional ARG)	Toggle superword-mode: a minor mode that treats **snake_case** and **kebab-case**/lisp-case as one word. In Emacs Lisp '-' and '_' are treated as part of words. • With a prefix argument, enable superword mode if ARG is positive, and disable it otherwise. • PEL provides the `<f12> M-p` key for the programming language modes where snake_case is popular (C, C++, Erlang, Python, etc…) and where **kebab-case**/lisp-case is used (Emacs Lisp).
• **By definitions/xref**	Move to the definition of the defun, defmacro, variable, etc… at point. See **Xref** for more information.		
Find definition of identifier at point See also: **Xref**	`M-.`	(**xref-find-definitions** IDENTIFIER)	Grab symbol at point and move cursor to its definition. • If there are more than one match, prompt in the *xref* buffer. • To search for a symbol entered manually, type `C-u M-.` • With dumb-jump this performs a search using ag, ripgrep or git grep if available.
Go back to where M-. was last issued	`M-,`	(**xref-pop-marker-stack**)	• Pop back to where M-. was last invoked. • Marker depth is controlled by the **xref-marker-ring-length** user option.
Find source code of function/ variable at point	• `<f12> .` • `M-<f12> .`	(**pel-find-thing-at-point**)	Find source code of function or variable at point. • Open in current window unless a `C-u` prefix is supplied as IN-OTHER-WINDOW in which case it opens inside the other window.
	`<f11> SPC l .`	🚧 The `M-.` key, part of the cross-reference support, is better for most purpose and it allows going back to the original location, which this one does but only via the mark ring. This command might be removed. **TODO:** more investigation needed.	
• **using elisp-refs** **Find references in code already loaded**	The **elisp-refs** commands find reference of code **already loaded** (⚠). It's useful to look into a problem because it will only find reference to code that could have been executed, but it will not find reference to any code that has not been loaded yet. 📦 Requires the **elisp-refs** external package. PEL activates it when **pel-use-elisp-refs** or **pel-use-helpful** user-options are set.		
Reference to function	`<f12> r f`	(**elisp-refs-function** SYMBOL &optional PATH-PREFIX)	Display all the references to function SYMBOL, in all loaded elisp files. • If called with a prefix, prompt for a directory to limit the search.
Reference to macro	`<f12> r m`	(**elisp-refs-macro** SYMBOL &optional PATH-PREFIX)	Display all the references to macro SYMBOL, in all loaded elisp files. • If called with a prefix, prompt for a directory to limit the search.
Reference to variable	`<f12> r v`	(**elisp-refs-variable** SYMBOL &optional PATH-PREFIX)	Display all the references to variable SYMBOL, in all loaded elisp files. • If called with a prefix, prompt for a directory to limit the search.
Reference to special form	`<f12> r s`	(**elisp-refs-special** SYMBOL &optional PATH-PREFIX)	Display all the references to special form SYMBOL, in all loaded elisp files. • If called with a prefix, prompt for a directory to limit the search.
Reference to symbol	`<f12> r o`	(**elisp-refs-symbol**SYMBOL &optional PATH-PREFIX)	Display all the references to SYMBOL, in all loaded elisp files. • If called with a prefix, prompt for a directory to limit the search.

• **To next/previous top-level forms**	Move to beginning /end of S-expression forms. Jump over comments. Can be defun, defer, defconst, defmacros, free-from S-exp, etc…
	The following `**beginning-of-defun**' and `**end-of-defun**' are standard Emacs commands. They have limitations:
	• They only navigate across any top-level form.
	  • They do not discriminate between a **defun**, a **defmacro** or even an **unless** form or any other top-level form.
	  • They do not skip doc-strings unless you set **open-paren-in-column-0-is-defun-start** user option to ignore '(' in strings.
	• PEL provides an additional commands, complementing the standard Emacs commands:
	  • **pel-beginning-of-next-defun** which moves forward to the beginning of the next form
	  • **pel-end-of-previous-defun** which moves backward to the end of the previous top-level form		

Change defun navigation functions (toggle between Emacs default and PEL's)	• `<f12> M-N`
	• `M-<f12> M-N`
	
	`<f11> SPC l M-N`
		(**pel-toggle-paren-in-column-0-is-defun-start**)	Toggle interpretation of a paren in column 0 and display new behaviour.
			• It toggles standard Emacs `**open-paren-in-column-0-is-defun-start**' user option, between:
			  • Interpret '(' in column 0 as always stating a defun (even in strings) - the default.
			  • Ignore '(' in strings. A '(' in column 0 is not automatically interpreted as starting a defun.

Backward to beginning of defun See also: **Navigation**	• `C-M-a` • `C-M-<home>` • `<f6> <up>`	(**beginning-of-defun** &optional ARG)	Move backward to the beginning of a top-level form: function definition, macros, etc… • With ARG, do it that many times. Negative ARG means move forward to the ARGth following beginning of defun. ☛ Shift marking is available in graphics mode, not in terminal mode (for `C-M-a` and `C-M-<home>`). It's always available for `<f6> <up>` : hold Shift after typing `<f6>`.
	☝ By default Emacs treats all opening parenthesis character in the first column as a defun. • This causes this function to stop at function definition inside strings. • The behaviour can be changed by setting the open-paren-in-column-0-is-defun-start user option to nil. • PEL provides **pel-toggle-paren-in-column-0-is-defun-start** to toggle that user option. You can also change it dynamically with `<f12> M-N`. ⚠ Moves to beginning of next function of the same nesting level of the current location. It skips the functions and methods that are more deeply nested.		
Forward to end of defun See also: **Navigation**	• `<f12> <right>` • `M-<f12> <right>` • `C-M-e` • `C-M-<end>` • `<f6> <right>`	(**end-of-defun** &optional ARG)	Move forward to next end of defun. With argument, do it that many times. Negative argument -N means move back to Nth preceding end of defun. ☛ Shift marking is available in graphics mode, not in terminal mode (for `C-M-e` and `C-M-<end>`). `<f6> <right>` and `<f12> <right>` support Shift-marking in terminal mode. ⚠ This command moves to the end of the next **top-level** function or class.
Forward to start of next top-level form	`<f6> <down>`	(**pel-beginning-of-next-defun** &optional SILENT DONT-PUSH_MARK)	Move forward to the beginning of the next top-level form: function definition, macros, etc.. • Beeps if does not find beginning of next function unless SILENT is non-nil. • If the beginning of next function is found, push the start location to the mark ring unless DONT-PUSH_MARK is non-nil. Move back to previous position with `M-`` or `<f6><f6>`. ☛ Shift marking is available with `<f6> <down>` : hold Shift after typing `<f6>`.
	☝ This command is generic and for Emacs Lisp, moves to the beginning of the next top-level form. • It also complements what end-of-defun does. It moves forward but to the beginning of the function definition, which is often what users expect. ☝ By default Emacs treats all opening parenthesis character in the first column as a defun. • This causes this function to stop at function definition inside strings. • The behaviour can be changed by setting the open-paren-in-column-0-is-defun-start user option to nil. • PEL provides **pel-toggle-paren-in-column-0-is-defun-start** to toggle that user option. You can also change it dynamically with `<f12> M-N`.		
Backward to end of previous defun	• `<f12> <left>` • `M-<f12> <left>` `<f6> <left>`	(**pel-end-of-previous-defun** &optional SILENT DONT-PUSH_MARK)	Move backwards to the end of the previous top-level form. • Beeps if does not find end of previous function unless SILENT is non-nil. • If the end of previous function is found, push the start location to the mark ring unless DONT-PUSH_MARK is non-nil. Move back to previous position with `M-`` or `<f6><f6>`. ☛ Shift marking is available.

Description	Keystroke	Function	Note
• **To next/previous selected S-expression form or defun or …** ★★	Move to beginning /end of specified S-expression forms. Jump over comments and docstrings. Can be defun, defer, defconst, defmacros, free-from S-exp, groups of them, etc… ☝ PEL provides the following powerful commands: **pel-elisp-beginning-of-next-form** and **pel-elisp-beginning-of-previous-forms**. • Their behaviour depends on the value of the **pel-elisp-target-forms**, **pel-elisp-user-specified-targets** and **pel-elisp-user-specified-targets2** user-options, as well as their corresponding global or buffer-local values if they exist. • The user options give you the ability to select the type of targets. You can either select the standard behaviour (target the top level forms), or use one of the other 7 types of targets. These include moving to top-level defun form, to any defun form, to defun, defmacro, defsubst, defalias, defadvice forms, to include the eieio forms, the variable definition forms or specify you own set of forms (and those can include the require and provide forms). • More information is available in the docstring of these user options. • When your buffer is using the Emacs-Lisp major mode, use the **`<f12> <f2>`** key sequence to open the relevant customization buffer which will allow you to see and change the persistent or current session settings. ☝ PEL also provides specialized versions of these commands: • **pel-elisp-beginning-of-next-defun** which moves to the beginning of next defun, **pel-elisp-beginning-of-previous-defun** to the previous defun. • **pel-elisp-to-name-of-next-defun** which moves to the *name* of the next defun, **pel-elisp-to-name-of-previous-defun** to the previous one. • **pel-elisp-to-name-of-next-form** which moves to the *name* of the next form, **pel-elisp-to-name-of-previous-form** to the previous one.		
Change target form for commands: ★★ • **<f12> <up>** • **<f12> <down>** • **<f12> C-<up>** • **<f12> C-<down>**	• **`<f12> M-n`** • **`M-<f12> M-n`**	(**pel-elisp-set-navigate-target-form** &optional GLOBALLY)	Select form navigation behaviour. Select the behaviour of the following navigation functions: • '**pel-elisp-beginning-of-next-form**' and • '**pel-elisp-beginning-of-previous-form**'.
	`<f11> SPC l M-n`	• Modifies the value of 'pel-elisp-target-forms' user-option only for the current buffer unless the GLOBALLY argument is non-nil, in which case it modifies the behaviour for all buffers. The change in behaviour does not persist across Emacs sessions. • For persistent change, open the customization buffer with **`<f12> <f2>`** , modify the value of the **pel-elisp-target-forms**, **pel-elisp-user-specified-targets** and **pel-elisp-user-specified-targets2** user-options and save the customize buffer.	
Forward to start of next definition form. ★★ ✓ Shift marking is available with **`<f12>`** **Configurable target:** • **all top-level forms** • **top-level defun** • **all defun** • **all defun, defsubst, defmacros, …** • **all variable definition forms: defvar, defconst, defcustom, defgroup, …** • **etc…**	• **`<f12> <down>`** • **`M-<f12> <down>`** **`<f11> SPC l <down>`**	(**pel-elisp-beginning-of-next-form** &optional N TARGET SILENT DONT-PUSH-MARK)	Move point forward to the beginning of next N top-level form. • The search is controlled by the value of '**pel-elisp-target-forms**' **pel-elisp-user-specified-targets** and **pel-elisp-user-specified-targets2** user options. That value can be changed for the current session, for all buffers or only for the current buffer by the command 'pel-elisp-set-navigate-target-form', bound to **`<f12> M-n`**. It can also be specified by the TARGET argument: specify one of the symbols valid for '**pel-elisp-target-forms**'.
	• The function skips over forms inside docstrings. If no valid form is found, don't move point, issue an error describing the failure unless SILENT is non-nil, in which case the function returns nil on error and non-nil on success. • On success, push original position on the mark ring unless DONT-PUSH-MARK is non-nil. Move back to previous position with **`M-``**. ☝ This command is the most flexible and can be configured to move like the next 2 commands. • It moves forward but to the beginning of the function definition, which is often what users of other editors expect. ☝ By default Emacs treats all opening parenthesis character in the first column as a defun: these are top-level forms. • You can change the behaviour: for example, to move to next define or any group of top-level or indented definition forms like defsubst, defmacro, defvar... • The behaviour is customizable (use **`<f12> <f2>`** then select the **pel-sexp-form-navigation** group to access the relevant user-options: **pel-elisp-target-forms**' , '**pel-elisp-user-specified-targets**' and '**pel-elisp-user-specified-targets2**'. Save the customization to make it persistent across Emacs sessions. • You can also control the values of these 2 user-options for all buffers or for each buffer separately: • You can change the values of these variables for a specific buffer or all buffers not yet configured by using the **`<f12> M-n`** command. • It's possible to set up a buffer to use the **`<f12> <down>`** key sequence to move to the next defun only or any top-level form, or some other selection or s-expression forms. • Or define your own selection in **pel-elisp-user-specified-targets** and '**pel-elisp-user-specified-targets2**' user-options, then activate them only for a buffer with **`<f12> M-n 8`** key sequence. ☝ To count & display # selected forms forward: use a large numeric argument to force a failure: the error message shows number of instances found. ☝ All of these commands push the point in the mark stack: use **`M-``** or **`<f6><f6>`** to move back to where the point was before the command was issued.		
Forward to the name of the next form definition	• **`<f12> C-<down>`** • **`M-<f12> C-<down>`**	(**pel-elisp-to-name-of-next-form** &optional N)	Move point to the name of next N defun form - at any level. • Skip over forms located inside docstrings. Leave point on the first character of the form name. • Move back to previous position with **`M-``** or **`<f6><f6>`**.
Forward to beginning of next defun form ✓ Shift marking is available with **`<f12> M-<down>`**	• **`<f12> M-<down>`** • **`<f12> f n`** • **`M-<f12> f n`** **`<f11> SPC l f n`**	(**pel-elisp-beginning-of-next-defun** &optional N)	Move point to the name of the next defun form, whether it is top-level or indented. • The function skips over forms inside docstrings. • Move back to previous position with **`M-``** or **`<f6><f6>`**. • This uses **pel-elisp-beginning-of-next-form** specifying 'defun-forms as target type.
Forward to the name of the next defun definition	• **`<f12> C-<M-down>`** • **`M-<f12> C-<M-down>`**	(**pel-elisp-to-name-of-next-defun** &optional N)	Move point to the name of next N defun form - at any level. • Skip over forms located inside docstrings and other types of forms. Leave point on first character of defun name. • Move back to previous position with **`M-``** or **`<f6><f6>`**.
Backward to start of previous definition form ★★ ✓ Shift marking is available **`<f12> <up>`** **Configurable target:** • **all top-level forms** • **top-level defun** • **all defun** • **all defun, defsubst, defmacros, …** • **all variable definition forms: defvar, defconst, defcustom, defgroup, …** • **etc…**	• **`<f12> <up>`** • **`M-<f12> <up>`** **`<f11> SPC l <up>`**	(**pel-elisp-beginning-of-previous-form** &optional N TARGET SILENT DONT-PUSH-MARK)	Move point backward to the beginning of previous N top-level form. • The search is controlled by the value of '**pel-elisp-target-forms**' user option. That value can be changed for the current session, for all buffers or only for the current buffer by the command 'pel-elisp-set-navigate-target-form', bound to **`<f12> M-n`**. It can also be specified by the TARGET argument: specify one of the symbols valid for '**pel-elisp-target-forms**'.
	• The function skips over forms inside docstrings. If no valid form is found, don't move point, issue an error describing the failure unless SILENT is non-nil, in which case the function returns nil on error and non-nil on success. • Move back to previous position with **`M-``** or **`<f6><f6>`**. ☝ This command is the most flexible and can be configured to move like the next 2 commands. • It moves backward but to the beginning of the function definition, which is often what users of other editors expect. ☝ By default Emacs treats all opening parenthesis character in the first column as a defun: these are top-level forms. • You can change the behaviour: for example, to move to next define or any group of top-level or indented definition forms like defsubst, defmacro, defvar, etc.. • The behaviour is customizable (use **`<f12> <f2>`** then select the **pel-sexp-form-navigation** group to access the relevant user-options: **pel-elisp-target-forms**', '**pel-elisp-user-specified-targets**' and '**pel-elisp-user-specified-targets2**'. The customization can be saved and then become persistent across Emacs sessions. • You can also control the values of these 2 user-options for all buffers or for each buffer separately: • You can change the values of these variables for a specific buffer or all buffers not yet configured by using the **`<f12> M-n`** command. • It's possible to set up a buffer to use the **`<f12> <up>`** key sequence to move to the previous defun only or any top-level form, or some other selection or s-expression forms. • Or define your own selection in **pel-elisp-user-specified-targets** and '**pel-elisp-user-specified-targets2**' user-options, then activate them only for a buffer with **`<f12> M-n 8`** key sequence. ☝ To count & display # selected forms backward: use a large numeric argument to force a failure: the error message shows # instances found.		
Backward to the name of the previous form definition	• **`<f12> C-<up>`** • **`M-<f12> C-<up>`**	(**pel-elisp-to-name-of-previous-form** &optional N)	Move point to the name of previous N defun form - at any level. • Skip over forms located inside docstrings. Leave point on the first character of the form name. • Move back to previous position with **`M-``** or **`<f6><f6>`**.
Backward to beginning of previous defun form ✓ Shift marking is available with **`<f12> M-<up>`**	• **`<f12> M-<up>`** • **`<f12> f p`** • **`M-<f12> f p`** • **`<f11> SPC l f p`**	(**pel-elisp-beginning-of-previous-defun** &optional N)	Move point to the name of the previous defun form, whether it is top-level or indented. • The function skips over forms inside docstrings. • On success, push original position on the mark ring unless DONT-PUSH-MARK is non-nil. • Move back to previous position with **`M-``** or **`<f6><f6>`**. • Uses **pel-elisp-beginning-of-previous-form** specifying 'defun-forms as target type.
Backward to the name of the previous defun definition	• **`<f12> C-<M-up>`** • **`M-<f12> C-<M-up>`**	(**pel-elisp-to-name-of-previous-defun** &optional N)	Move point to the name of previous N defun form - at any level. • Skip over forms located inside docstrings and other types of forms. Leave point on first character of defun name. • Move back to previous position with **`M-``** or **`<f6><f6>`**.

Description	Keystroke	Function	Note
• **By S-Expression form**	Move across forms (S-expressions in Lisp).		
• **By List element**	• Move backward to the beginning or forward to the end of a S-expression form		
Backward block/list See also: **Σ Navigation**	**C-M-p**	(**backward-list** &optional ARG)	Move backward across one balanced group of parentheses. • This command will also work on other parentheses-like expressions defined by the current language mode. • With ARG, do it that many times. • Negative arg -N means move forward across N groups of parentheses. • This command assumes point is not in a string or comment. • **C-M-p** : ☛ Shift marking is available in graphics mode, not in terminal mode.
Move block backward See also: • **Σ Navigation**	• **C-M-b** • **C-M-<left>** • **C-[C-b** • **Esc C-b** • **Esc C-<left>** ⚠️	(**backward-sexp** &optional ARG)	Move backward across one balanced expression (sexp). • With ARG, do it that many times. Negative arg -N means move forward across N balanced expressions. This command assumes point is not in a string or comment. • **C-M-b** : ☛ Shift marking is available in graphics mode, not in terminal mode. • **C-M-<left>** : ☛ Shift marking works with this command. ❖ **C-M-<left>** does not work on Windows, but **H-<left>** works.
	⚠️ With PEL: if you want to use **Esc C-<left>** binding you must ensure that **pel-windmove-on-esc-cursor** user option is set to nil. 🐧 Several Linux distros map **C-M-<left>** to desktop workspace operation. In that case you can either use another key binding or change Linux key binding in Systems->settings->keyboard->shortcuts to prevent it from using that key sequence.		
Forward block/list See also: **Σ Navigation**	**C-M-n**	(**forward-list** &optional ARG)	Move forward across one balanced group of parentheses. • This command will also work on other parentheses-like expressions defined by the current language mode. • With ARG, do it that many times. • Negative arg -N means move backward across N groups of parentheses. • This command assumes point is not in a string or comment. • **C-M-n** : ☛ Shift marking is available in graphics mode, not in terminal mode.
Move block forward See also: • **Σ Navigation**	• **C-M-f** • **C-M-<right>** • **C-[C-f** • **Esc C-f** • **Esc C-<right>** ⚠️	(**forward-sexp** &optional ARG)	Move forward across one balanced expression (sexp). • With ARG, do it that many times. Negative arg -N means move backward across N balanced expressions. This command assumes point is not in a string or comment. • **C-M-f** : ☛ Shift marking is available in graphics mode, not in terminal mode. • **C-M-<right>** : ☛ Shift marking works with this command. ❖ **C-M-<right>** does not work on Windows, but **H-<right>** does.
	⚠️ With PEL: if you want to use **Esc C-<right>** binding you must ensure that **pel-windmove-on-esc-cursor** user option is set to nil. 🐧 Several Linux distros map **C-M-<right>** to desktop workspace operation. In that case you can either use another key binding or change Linux key binding in Systems->settings->keyboard->shortcuts to prevent it from using that key sequence.		
• **in/out of lists**	• Move in and out of list nested levels.		
Backward Up/outside sexp hierarchy See also: • **Σ Navigation**	• **C-M-u** • **C-M-<up>** • **C-[C-u** • **Esc C-u** • **Esc C-<up>** ⚠️	(**backward-up-list** &optional ARG ESCAPE-STRINGS NO-SYNTAX-CROSSING)	Move backward out of one level of parentheses. • This command will also work on other parentheses-like expressions defined by the current language mode. With ARG, do this that many times. A negative argument means move forward but still to a less deep spot. • ⚠️ With PEL: if you want to use **Esc C-<up>** binding you must ensure that **pel-windmove-on-esc-cursor** user option is set to nil. • **C-M-u** : ☛ Shift marking is available in graphics mode, not in terminal mode. • **C-M-<up>** : ☛ Shift marking works with this command. ❖ **C-M-<up>** does not work on Windows, but **H-<up>** does.
Forward Up/outside sexp hierarchy See also: **Σ Navigation**	**C-M-]**	(**up-list** &optional ARG ESCAPE-STRINGS NO-SYNTAX-CROSSING)	Move forward out of one level of parentheses. • This also works on other parentheses-like expressions defined by the current language mode. • With ARG, do this that many times. A negative argument means move backward but still to a less deep spot. • If ESCAPE-STRINGS is non-nil (as it is interactively), move out of enclosing strings as well. • If NO-SYNTAX-CROSSING is non-nil (as it is interactively), prefer to break out of any enclosing string instead of moving to the start of a list broken across multiple strings. On error, location of point is unspecified.
Forward Down/inside sexp/ block See also: • **Σ Navigation**	• **C-M-d** • **C-M-<down>** • **C-[C-d** • **Esc C-d** • **Esc C-<down>** ⚠️	(**down-list** &optional ARG)	Move forward down one level of parentheses. • This also works on other parentheses-like expressions defined by the current language mode. • With ARG, do this that many times. A negative argument means move backward but still go down a level. • This command assumes point is not in a string or comment. • ⚠️ With PEL: if you want to use **Esc C-<down>** binding you must ensure that **pel-windmove-on-esc-cursor** user option is set to nil. • **C-M-d** : ☛ Shift marking is available in graphics mode, not in terminal mode. • **C-M-<down>** : ☛ Shift marking works with this command. ❖ **C-M-<down>** does not work on Windows, but **H-<down>** does.
• **By sentences**	Move to beginning /end of statement of comment sentence. • The variable 'sentence-end' is a regular expression that matches ends of sentences. Useful in comments. In code it moves to the beginning or end of a definition form (defun, defmacro, etc…)		
Move to beginning of sentence or form	**M-a**	(**backward-sentence** &optional ARG)	Move backward to start of sentence. With arg, do it arg times. ☛ Shift marking works with this command.
Move forward to end of sentence or form	**M-e**	(**forward-sentence** &optional ARG)	Move forward to next end of sentence. With argument, repeat. With negative argument, move backward repeatedly to start of sentence. ☛ Shift marking works with this command.

Description	Keystroke	Function	Note
Macro Expansion	The macrostep package provides the macrostep-expand command that expands the macro code in the buffer (temporary turning buffer in read-only mode). 📦 This requires the **macrostep** package. Under PEL, activated with ***pel-use-macrostep*** user option.		
Expand macro form code with macrostep	• **<f12> M-m** • **M-<f12> M-m** **<f11> SPC l M-m**	(**macrostep-expand** &optional TOGGLE-SEPARATE-BUFFER)	Expand the macro form following point by one step. • Enters 'macrostep-mode' if it is not already active, making the buffer temporarily read-only. If macrostep-mode is active and the form following point is not a macro form, search forward in the buffer and expand the next macro form found, if any. • With prefix argument, the expansion is displayed in a separate buffer instead of inline in buffer. Setting 'macrostep-expand-in-separate-buffer' to non-nil swaps these two behaviors.

macrostep-mode keys	Progressively expand macro forms with **e**, collapse them with **c**, and move back and forth with **n** and **p**.
	Use **q** or collapse all visible expansions to quit and return to normal editing.
	`key --- binding ------- = macrostep-expand c macrostep-collapse e macrostep-expand n macrostep-next-macro p macrostep-prev-macro`
	`key --- binding ------- q macrostep-collapse-all u macrostep-collapse DEL macrostep-collapse C-c C-c macrostep-collapse-all C-M-i macrostep-prev-macro`		

Code Analysis	The commands below are used to **analyze** the Emacs Lisp code.		
Check validity of parentheses (or quotes, braces, brackets) (See also: 𝔓𝔩 Common Lisp)	• **<f12>)** • **M-<f12>)** • **<f12> a)** • **M-<f12> a)** **<f11> SPC l a)**	(**check-parens**)	Check for unbalanced parentheses in the current buffer. • More accurately, check the narrowed part of the buffer for unbalanced expressions ("sexps") in general. This is done according to the current syntax table and will find unbalanced brackets or quotes as appropriate. (See Info node '(emacs)Parentheses'.) If imbalance is found, an error is signaled and point is left at the first unbalanced character.
ELint the code in current buffer	• **<f12> a b** • **M-<f12> a b** **<f11> SPC l a b**	(**pel-lint-elisp-file**)	Run lint on Emacs Lisp file in current buffer. • This uses Elint. • This will open all Emacs Lisp files referred by the current file (via calls such as require calls) but also the files used by Emacs, to complete the lint analysis.
Analyze the style and documentation of code in current buffer	• **<f12> a d** • **M-<f12> a d** **<f11> SPC l a d**	(**checkdoc**)	Interactively check the entire buffer for style errors. • The current status of the check will be displayed in a buffer which the users will view as each check is completed. • When errors are detected the analysis pauses and the user can enter **recursive edit** mode to correct the current style error and then resume the analysis by exiting the recursive edit with **C-M-c**.
ELint a specific Emacs Lisp file.	• **<f12> a f** • **M-<f12> a f** **<f11> SPC l a f**	(**elint-file** FILE)	Lint the file FILE. • Emacs prompts for the file name.
Analyze the sexp at point and print a declare message if define is pure and/or side-effect-free	• **<f12> a s** • **M-<f12> a s** **<f11> SPC l a s**	(**pel-elcode-properties-of-sexp-at-point**)	Analyze the sexp at point and print a message that shows the declare form to identify whether the sexp is pure, side-effect-free or side-effect-free and error-free. ☝ Use this to help you add the (declare (pure t) (side-effect-free error-free)) form for a define form or just to check if the sexp at point is pure and/or side-effect-free.
ParInfer EDiff **Diff current code before/.after ParInfer modifications** See also: **Diff & Merge**	• **<f12> a D** • **M-<f12> a D** **<f11> SPC l a D**	(**parinfer-diff**)	Diff current code and the code after applying Indent Mode in Ediff. Use this to browse and apply the changes. 📦 Requires parinfer external package. PEL activates it when **pel-use-parinfer** is set to **t**.
• **package-lint**	📦 This requires the **package-lint** external package activated by **pel-use-package-lint**.		
Lint Emacs Lisp Package	**<f12> a p p** **<f11> SPC l a p p**	(**(package-lint-current-buffer)**)	Display lint errors and warnings for the current buffer.
Show history of an Emacs Lisp symbol	**<f12> a p h** **<f11> SPC l a p h**	(**package-lint-describe-symbol-history** SYM)	Show the version history of SYM, if any. Prompt for symbol, use symbol at point. ☝ Use this to show when an emacs lisp variable or function was added, or modified in Emacs.
• **Flycheck Analysis** **See also:** **SyntaxCheck**	The following extra key bindings are available when flycheck is active. Toggle flycheck-mode on/off with **<f11> ! !** 📦 The **flycheck** external package is activated by PEL when the **pel-use-flycheck .** ☝ The **flycheck** checkers for Emacs Lisp includes **emacs-lisp-checkdoc**, a very useful utility now integrated with Emacs.		
Toggle flycheck mode for current buffer	**<f11> ! !**	(**flycheck-mode** &optional ARG)	Activate (or deactivate) **flycheck** minor-mode for the current buffer. • Once **flycheck** minor mode is active, several other key bindings become available. They are described in the **SyntaxCheck** page. It includes the key binding shown in the next row.
Show error list for current buffer	• **C-c ! l** • **<f11> ! l**	(**flycheck-list-errors**)	Show the error list for the current buffer.
• **flycheck-package**	📦 This requires the **flycheck-package** external package activated by **pel-use-flycheck-package**.		
Activate package-lint flycheck checker	**<f12> a p !** **<f11> SPC l a p !**	(**flycheck-package-setup**)	Setup flycheck-package. Add 'flycheck-emacs-lisp-package' to '**flycheck-checkers**'. ☝ See **SyntaxCheck** for commands to see active checkers and to disable checkers.
relint — Regular Expression Lint **See also:** **Search/Replace**	Analyze the validity of the regular expressions inside Emacs Lisp code stored inside the current Emacs Lisp buffer, an Emacs Lisp file or, all Emacs Lisp files inside a directory tree. From the *relint* buffer press **g** to re-run the same checks. • The package can also used in a script to analyze regular expressions using Emacs batch invocation. 📦 Requires the **relint** external package. PEL installs and activates it when the **pel-use-relint** user-option is set to **t**.		
Lint regular expressions in current buffer	**<f12> a l b** **<f11> s x M-l b**	(**relint-current-buffer**)	Scan the current buffer for regexp errors. ⚠️ The buffer must be in emacs-lisp-mode.
Lint regular expressions in specified file	**<f12> a l f** **<f11> s x M-l f**	(**relint-file** FILE)	Scan FILE, an elisp file, for regexp-related errors. • Prompts for Emacs Lisp file.
Lint regular expressions in specified directory	**<f12> a l d** **<f11> s x M-l d**	(**relint-directory** DIR)	Scan all *.el files in DIR for regexp-related errors. Prompts for the directory. • Scans directory tree: all Emacs Lisp files in the specified directory all all sub-directories , recursively.
List library dependencies	📦 Requires **elisp-depend** activated by **pel-use-elisp-depend**		
Insert dependencies	**<f12> a M-d c**	(**elisp-depend-insert-comment**)	Insert a block of 'sym' statements into an elisp file.
Print list of dependencies	**<f12> a M-d d**	(**elisp-depend-print-dependencies** &optional BUILT-IN)	Print library dependencies of the current buffer. • With prefix argument, don't include built-in libraries. • Every library with a parent directory in '**elisp-depend-directory-list**' is considered built-in.
Insert require forms for dependencies	**<f12> a M-d r**	(**elisp-depend-insert-require**)	Insert a block of (require sym) or 'autoload statements into an elisp file.

Description	Keystroke	Function	Note
Generate Elisp code dependency Graphviz Dot diagrams.	📦 Requires **elisp-depmap** activated by **pel-use-elisp-depmap** For GUI mode only. ⚠️ These operations are all CPU intensive and block Emacs while performing the analysis and that can be very very long.... Use a dedicated process.		
Generate an directed graph	`<f12> a M-D d`	(**elisp-depmap-graphviz-digraph** &optional SHUFFLE)	Make a dot file representation of all definitions and references. • Optionally set INDENT-WIDTH which is 2 by default. • If SHUFFLE gives a random seed (default 0) to shuffle subgraph cluster layouts. • Useful if the project contains only one file. Each node is a function coloured by file, and shaped by variable or function type (e.g. setq, defvar, defcustom, defun, defsubst, defmacro). Each edge indicates that a function is linked to another. Border width indicates the size of the function.
Generate a un-directed graph	`<f12> a M-D g`	(**elisp-depmap-graphviz**)	Make basic dot file of top level definitions and their references. • Useful if the project contains multiple files, where functions from the same file are contained under the same subgraph, and an arrow from a source node/function to a target node/function indicates that the source node was called by the target.
Generate a summary table	`<f12> a M-D s`	(**elisp-depmap-makesummarytable**)	Make a summary org table of variables and references to them.
Compiling	The commands below are used to **compile** the Emacs Lisp source code into byte code (.elc files) and navigate across the byte-compilation errors. When errors are detected, they are shown in a buffer. You can also click on the error links or type return on them to move point to the code error location.		
Byte-compile current buffer visited file	• `<f12> c b` • `M-<f12> c b` • `<f12> M-c` • `M-<f12> M-c` `<f11> SPC l c b`	(**pel-byte-compile-file-and-load**)	Byte compile the currently visited elisp file and load the resulting byte-compiled code.
Byte-compile current buffer visited file and optionally load it. Emacs >= 29.1	`C-c C-f`	(**elisp-byte-compile-file** &optional LOAD)	Byte compile the file the current buffer is visiting. • If LOAD is non-nil, load the resulting .elc file. When called interactively, this is the prefix argument.

Byte-compile complete directory of Emacs Lisp files	• `<f12> c d`
	• `M-<f12> c d`
	`<f11> SPC l c d`
		(**byte-recompile-directory** DIRECTORY &optional ARG FORCE)	Recompile every '.el' file in DIRECTORY *that needs recompilation*.
			• This happens when a '.elc' file exists but is older than the '.el' file. Files in subdirectories of DIRECTORY are processed also.
			• It's possible to specify the first argument interactively (but not the second):
			  • If the '.elc' file does not exist, normally this function *does not* compile the corresponding '.el' file. However, if the prefix argument ARG is 0, that means do compile all those files. A nonzero ARG means ask the user, for each such '.el' file, whether to compile it. A nonzero ARG also means ask about each subdirectory before scanning it.
			  • If the third argument FORCE is non-nil, recompile every '.el' file that already has a '.elc' file.
			☝️ If you upgrade or change version of Emacs you may want to byte recompile all files even if the .elc files exist and are newer than their corresponding .el file. In that case you must delete the .elc files first and then use the **`C-u 0`** prefix.

Byte compile specified Emacs Lisp file	• `<f12> c f` • `M-<f12> c f` `<f11> SPC l c f`	(**byte-compile-file** FILENAME &optional LOAD)	Compile a file of Lisp code named FILENAME into a file of byte code. • Emacs prompts for the filename. • The output file's name is generated by passing FILENAME to the function 'byte-compile-dest-file' (which see). • With prefix arg (noninteractively: 2nd arg), LOAD the file after compiling.
Move to next compile error	• `C-x ` `  • `M-g n` • `M-g M-n`	(**next-error** &optional ARG RESET)	A prefix ARG specifies how many error messages to move; • negative means move back to previous error messages. • Just **`C-u`** as a prefix means reparse the error message buffer and start at the first error. ⚠️ This only shows the result of compilations; it does not report Flycheck reported errors. To use it you must byte-compile the file first.
Move to previous compile error	• `M-g p` • `M-g M-p`	(**previous-error** &optional N)	Prefix arg N says how many error messages to move backwards (or forwards, if negative). ⚠️ This only shows the result of compilations; it does not report Flycheck reported errors. To use it you must byte-compile the file first.
Disassemble a function	• `<f12> c a` • `M-<f12> c a` `<f11> SPC l c a`	(**disassemble** OBJECT &optional BUFFER INDENT INTERACTIVE-P)	Print disassembled code for OBJECT in (optional) BUFFER. • Prompts for object, normally a function. Supports tab completion. • OBJECT can be a symbol defined as a function, or a function itself (a lambda expression or a compiled-function object). • If OBJECT is not already compiled, we compile it, but do not redefine OBJECT if it is a symbol.
Regression Testing See also: **ERT**	The Emacs Lisp Regression Testing (ERT) is what you use to write regression tests for Emacs Lisp. It is better described in the **ERT** page. With PEL, the easiest is to open a ERT compliant test file and use **`<f12> t`** to byte-compile it and then run the tests.		
Run test interactively	`M-x ert`	(**ert** SELECTOR &optional OUTPUT-BUFFER-NAME MESSAGE-FN)	Run the tests specified by SELECTOR and display the results in a buffer. • SELECTOR works as described in 'ert-select-tests'. (Use **t** to run all tests, or name the test to execute. • OUTPUT-BUFFER-NAME and MESSAGE-FN should normally be nil; they are used for automated self-tests and specify which buffer to use and how to display message. • By default, the results are stored inside the *ert* buffer, opened in ERT-Results mode.
Byte Compile and run tests	`<f12> C-t C-t`	(**pel-run-ert**)	Byte compile and run ERT test on current buffer. • Prompts if the buffer needs to be saved first.

Description	Keystroke	Function	Note
Debugging Emacs Lisp	Emacs comes with 2 debuggers: 1. **debug** : built in Emacs, always available, uses the *Backtrace* buffer to show backtrace of execution. 2. **edebug**: source level debugger, shows the execution right inside the source code buffer.		

• **Debug**	There are several ways to debug using debug:
	• Instrument the code by placing a **`(debug)`** call acting as breakpoints into the code to inspect.
	  • Use the commands listed below to invoke or schedule the invocation of the debugger, or
	  • kill the Emacs process externally with: **`pkill -SIGUSR2 -i emacs`** which toggles debug-on-quit when Emacs is hung.
	🎛 Debugger customization user option variables that control the debugger behaviour:
	• **debug-on-error**:
	  • Non-nil means enter debugger if an error is signalled.
	  • Does not apply to errors handled by 'condition-case' or those matched by '**debug-ignored-errors**'.
	  • If the value is a list, an error only means to enter the debugger if one of its condition symbols appears in the list.
	  • When you evaluate an expression interactively, this variable is temporarily non-nil if '**eval-expression-debug-on-error**' is non-nil.
	  • The command '**toggle-debug-on-error**' toggles this.
	• **debug-on-next-call**:
	  • Non-nil means enter debugger before next 'eval', 'apply' or 'funcall'.
	• **debug-on-quit**:
	  • Non-nil means enter debugger if quit is signaled (**`C-g`**, for example). Does not apply if quit is handled by a 'condition-case'.
	• **inhibit-debugger**:
	  • Non-nil means never enter the debugger. Normally set while the debugger is already active, to avoid recursive invocations.		

☝ **To activate debug on error:**	When investigating an error in the Emacs Lisp code, activate the debugger by executing: **`M-: (setq debug-on-error t)`**		
Identify function to debug	• **`<f12> d f`** • **`M-<f12> d f`** **`<f11> SPC l d f`**	(**debug-on-entry** FUNCTION)	Request FUNCTION to invoke debugger each time it is called. • When called interactively, prompt for FUNCTION in the minibuffer. • This works by modifying the definition of FUNCTION. If you tell the debugger to continue, FUNCTION's execution proceeds. If FUNCTION is a normal function or a macro written in Lisp, you can also step through its execution. FUNCTION can also be a primitive that is not a special form, in which case stepping is not possible. Break-on-entry for primitive functions only works when that function is called from Lisp. • Use **`M-x cancel-debug-on-entry`** to cancel the effect of this command. • Redefining FUNCTION also cancels it.
Cancel debugging of function	• **`<f12> d F`** • **`M-<f12> d F`** **`<f11> SPC l d F`**	(**cancel-debug-on-entry** &optional FUNCTION)	Cancel the debugging of specified function: undo effect of **`M-x debug-on-entry`** on FUNCTION. • If FUNCTION is nil, cancel debug-on-entry for all functions. • When called interactively, prompt for FUNCTION in the minibuffer. • To specify a nil argument interactively, exit with an empty minibuffer.
Activate/disable debugger on error	• **`<f12> d !`** • **`M-<f12> d !`** **`<f11> SPC l d !`**	(**toggle-debug-on-error** &optional INTERACTIVELY)	Toggle whether to enter Lisp debugger when an error is signaled. • In an interactive call, record this option as a candidate for saving by "Save Options" in Custom buffers.
Activate/disable debugger on quit	• **`<f12> d )`** • **`M-<f12> d )`** **`<f11> SPC l d )`**	(**toggle-debug-on-quit** &optional INTERACTIVELY)	Toggle whether to enter Lisp debugger when **`C-g`** is pressed. • In an interactive call, record this option as a candidate for saving by "Save Options" in Custom buffers.
Invoke debugger when variable is modified	• **`<f12> d v`** • **`M-<f12> d v`** **`<f11> SPC l d v`**	(**debug-on-variable-change** VARIABLE)	Prompt for VARIABLE. Trigger a debugger invocation when VARIABLE is changed. • This works by calling 'add-variable-watcher' on VARIABLE. If you quit from the debugger, this will abort the change (unless the change is caused by the termination of a let-binding). • The watchpoint may be circumvented by C code that changes the variable directly (i.e., not via 'set'). Changing the value of the variable (e.g., 'setcar' on a list variable) will not trigger watchpoint. • Use **`<f12> d V`** to cancel the effect of this command. Uninterning VARIABLE or making it an alias of another symbol also cancels it.
Cancel debugger invocation on modified variable	• **`<f12> d V`** • **`M-<f12> d V`** **`<f11> SPC l d V`**	(**cancel-debug-on-variable-change** &optional VARIABLE)	Prompt for VARIABLE. Undo effect of **`<f12> d v`** on VARIABLE. • If VARIABLE is nil, cancel debug-on-variable-change for all variables. • To specify a nil argument interactively, exit with an empty minibuffer.
Debugger *Backtrace* buffer commands	When the debugger is invoked, a *Backtrace* buffer window opens which displays the Lisp stack. Each line represents a function call, the most recent at the top. With it it is possible to view pending Lisp expressions, check the value of variables and force functions to return specified values. The mode accepts the commands listed below. • Step through the debugger using **d** • Use **c** to skip over an evaluation • Use **e** to evaluate a variable of interest in the concept of the code, or: hit **RET** with the cursor over the variable to evaluate it • Sexp can be evaluating within the calling context. • Provide a sexp to evaluate to function debug, showing the value when the debugger is opened.		
Step through	**`d`**	(**debugger-step-through**)	Proceed, stepping through subexpressions of this expression. Enter another debugger on next entry to eval, apply or funcall.
Continue	**`c`**	(**debugger-continue**)	Continue code execution - leave the debugger. • This is not available when the debugger was invoked because of an error.
Jump	**`j`**	(**debugger-jump**)	Continue to exit from this frame, with all debug-on-entry suspended.
Show/Hide variable	**`v`**	(**debugger-toggle-locals**)	Show or hide local variables of the current stack frame.
Evaluate expression	**`e`**	(**debugger-eval-expression** EXP &optional NFRAME)	Eval an expression, in an environment like that outside the debugger. The environment used is the one when entering the activation frame at point.
Display and Record expression	**`R`**	(**debugger-record-expression** EXP)	Display a variable's value and record it in '*Backtrace-record*' buffer.
Return value	**`r`**	(**debugger-return-value** VAL)	Continue, specifying value to return. • This is only useful when the value returned from the debugger will be used, such as in a debug on exit from a frame.
Debug frame	**`b`**	(**debugger-frame**)	Request entry to debugger when this frame exits. • Applies to the frame whose line point is on in the backtrace. • **Break** when returning from current function, continuing execution for the body of the function.
Cancel Debug frame	**`u`**	(**debugger-frame-clear**)	Do not enter debugger when this frame exits. • Applies to the frame whose line point is on in the backtrace.
Quit	**`q`**	(**top-level**)	Quit the debugger. Abort pending operation. Close the window and return point to previous location.
List functions that have debug on entry	**`d`**	(**debugger-list-functions**)	Display a list of all the functions now set to debug on entry.

Description	Keystroke	Function	Note
• **EDebug**	Emacs edebug is a source level debugger, used within the Emacs Lisp source code. • It shows more than the stack frame, putting a cursor in the source code where the break point is located. • Edebug can be used to step though the code or not stop at all and gather execution coverage and frequency data. • Once EDebug stops at a breakpoint the key binding of the EDebug commands that can only be used within the buffer currently in edebug-mode. • ie. where EDebug is active. The Edebug key bindings are shown below in coral color. • Some of the commands can also be issued from other buffers with different key bindings (and those are show in black). • When an Emacs Lisp buffer has entered edebug-mode its mode line displays *Debugging*.		
Instrumenting for Edebug	To use EDebug, first instrument the function(s) you want the debugger to step into: • Put point within or just after the function definition and type one of **C-u C-M-x** or **<f12> d e**. • It is also possible to instrument all definitions in a buffer and even all forms in a buffer: it must be activated for that using **edebug-all-defs** or **edebug-all-forms**.		
To remove Edebug instrumentation on a function	If you no longer want the debugger to stop on a function, you must re-evaluate the function definition. This will remove the edebug instrumentation from the function definition and it will no longer stop in the debugger. Use one of the two methods to do that: • Move point right after the end of the function and execute **eval-last-sexp** by typing: **C-x C-e** • Move point inside the function and execute **eval-defun** by typing: **C-M-x**		
Instrument most forms for Edebug (with variable controlling behaviour)	**C-M-x**	(**eval-defun** EDEBUG-IT) — — — — — — — — — — (**edebug-eval-defun** EDEBUG-IT)	Evaluate the top-level form containing point or after point. • Without prefix argument: remove EDebug instrumentation for the function at point. • With **C-u** prefix argument: activate Edubug instrumentation for the function at point. The very first time (eval-defun t) is executed it loads edebug.el and advise eval-defun to edebug-eval-defun. • The following variables provide extra control: • If **edebug-all-defs** is non-nil, that inverts the meaning of the prefix argument: in that case C-M-x instruments the definition unless it has a prefix argument. Its default is nil. • If **edebug-all-defs** is non-nil, then the commands eval-region, eval-current-buffer and eval-buffer also instrument any definition they evaluate. • If **edebug-all-forms** control whether eval-region should instrument any form, even non-defining forms. This does not apply to loading or evaluation in the minibuffer.
Toggle all EDebug defun instrumentation	M-x edebug-all-defs	(**edebug-all-defs**)	Toggle edebugging of all definitions that could be done by eval-region, eval-current-buffer and eval-buffer.
Toggle instrumenting for EDebugging of all forms	M-x edebug-all-forms	(**edebug-all-forms**)	Toggle edebugging of all forms.
Instrument top level form (always) for Edebug	• **<f12> d e** • **M-<f12> d e** **<f11> SPC l d e**	(**edebug-defun**)	Evaluate the top level form point is in, stepping through with Edebug. • This is like 'eval-defun' except that it steps the code for Edebug before evaluating it. It displays the value in the echo area using 'eval-expression' (which see).
	• If you do this on a function definition such as a defun or defmacro, it defines the function and instruments its definition for Edebug, so it will do Edebug stepping when called later. It displays 'Edebug: FUNCTION' in the echo area to indicate that FUNCTION is now instrumented for Edebug. • If the current defun is actually a call to 'defvar' or 'defcustom', evaluating it this way resets the variable using its initial value expression even if the variable already has some other value. (Normally 'defvar' and 'defcustom' do not alter the value if there already is one.) • Instruments any top level form regardless of the value of edebug-all-defs and edebug-all-forms. edebug-defun is an alias for edebug-eval-top-level-form.		
Instrument one more definition	I	(**edebug-instrument-callee**)	Instrument the definition of the function or macro about to be called (just after point). ☛ This command is only available when EDebug is active. • Do this when stopped before the form or it will be too late. • One side effect of using this command is that the next time the function or macro is called, Edebug will be called there as well. • If the callee is a generic function, Edebug will instrument all the methods, not just the one which is about to be called. Return the list of symbols which were instrumented.
EDebug Help	Once EDebug is active, use **?** to get help; a description of all available commands is listed on the Help buffer.		
Help	?	(**edebug-help**)	Describe 'edebug-mode'. Print the list of available Edebug commands inside a Help buffer.
Edebug Execution Modes	Once function(s) are instrumented, simply execute the code you want to debug. Once the debugger has reached a breakpoint Emacs enter the edebug-mode and the commands listed below are available. A quick overview, taken from the edebug.el source code state: • Step through the code with **SPC**, • Mark breakpoint with **b**, • Go until a breakpoint is reached with **g**, • Quit execution with **q**. • Use **?** to to describe other commands. The following commands correspond to EDebug execution modes (EDebug ways of operating — not related to the concept of Emacs minor/major modes). The commands in the list below run the program more slowly or stop sooner than the commands later in the list.		
Stop	S	(**edebug-stop**)	Stop execution and do not continue. • Useful for exiting from trace or continue loop.
Step	• **SPC** • **C-c C-s** • **C-x C-a C-s** • **C-x X SPC**	(**edebug-step-mode**)	Proceed to next stop point.
Next	• **n** • **C-c C-n** • **C-x C-a C-n**	(**edebug-next-mode**)	Proceed to next 'after' stop point.
Trace	• **t** • **C-x X t**	(**edebug-trace-mode**)	Begin trace mode: pause (normally 1 second) at each EDebug stop point. • Pauses for 'edebug-sit-for-seconds' at each stop point. • The trace can be interrupted by any key (like a navigation key or one of the EDebug command keys).
Trace Fast	• **T** • **C-x X T**	(**edebug-Trace-fast-mode**)	Trace with no wait at each step. • Updates the display at each stop point, but does not pause. • The trace can be interrupted by any key (like a navigation key or one of the EDebug command keys).
Go	• **g** • **C-x X g**	(**edebug-go-mode** ARG)	Go, evaluating until break: run until next breakpoint. • With prefix ARG, set temporary break at current point and go.
Continue	• **c** • **C-x X c**	(**edebug-continue-mode**)	Begin continue mode: pause one second at each breakpoint and then continue. • Pauses for 'edebug-sit-for-seconds' at each break point.
Continue Fast	• **C** • **C-x X C**	(**edebug-Continue-fast-mode**)	Trace with no wait at each step. • Updates the display at each break point, but does not pause.
Go Nonstop	• **G** • **C-x X G**	(**edebug-Go-nonstop-mode**)	Go, evaluating without debugging (ignoring the breakpoints). • You can also use 'edebug-stop', or any editing command, to stop.

Description	Keystroke	Function	Note
Controlling EDebug Execution Mode	By default EDebug stops at the first instrumented function it encounters. It can also be configured to stop only at the first breakpoint or never (useful for gathering coverage data). This is controlled by the value of the *edebug-initial-mode*. The possible values are: • step (the default) • go • Go-nonstop • some other EDebug options The following function can be used to change this.		
Change initial execution mode.	• `C-x C-a RET` • `C-x C-a C-m`	(**edebug-set-initial-mode**)	Set the initial execution mode of Edebug. • The mode is requested via the key that would be used to set the mode in edebug-mode. • This command prompts for the execution mode key, one of the single letters commands listed in the section above: SPC, n, t, T, g, c, C or G.
Edebug Jumping	The following commands execute until execution reach the specified location (or reach another breakpoint before). Except for step in they all create a temporary breakpoint for the intended destination. The commands, can, however, fail in case of nonlocal exit, bypassing reaching the temporary breakpoint. • The **`f`**, **`o`** and **`h`** commands display "Break" and pause for *edebug-sit-for-seconds* before showing the result of the form just evaluated. Setting this variable to nil suppresses this delay.		
Jump forward sexp	`f`	(**edebug-forward-sexp** ARG)	Proceed from the current point to the end of the ARGth sexp ahead. • If there is no Arg, jump forward 1 sexp • If there are not ARG sexps ahead, then do 'edebug-step-out'. ☛ If point is not located where the next step is, you can type **`w`** to move point there, before typing **`f`**. ⚠ Note that you must ensure that execution will go to the specified number of sexp, as it may not be the case if there are any conditional forms in the path.
Jump: step in	`i`	(**edebug-step-in**)	Step into the definition of the function, macro or method about to be called. • This first does 'edebug-instrument-callee' to ensure that it is instrumented. Then it does 'edebug-on-entry' and switches to 'go' mode. ☛ Once you step in a function with **i** it remains instrumented and will cause a stop upon future execution within the same Edebug session. To prevent this, simply re-evaluate the definition of that function to deinstrument it.
Jump: step out	`o`	(**edebug-step-out**)	Proceed from the current point to the end of the containing sexp. • If there is no containing sexp that is not the top level defun, go to the end of the last sexp, or if that is the same point, then step. • If the containing sexp is a function definition, this command continues until just before the last sexp in the definition. If it is already there, it returns from the function then stops. Essentially this command does not exit the currently executing function unless point is already positioned after its last sexp.
Goto here	`h`	(**edebug-goto-here**)	Proceed to first stop-point at or after current position of point. • Use this to execute up until a specific point (such as inside a specific condition) to see if execution gets there or when running a loop to see a specific value. • This does not set any breakpoint, so if you want to run again up to this location you can type **h** again on the same location.
EDebug Breakpoints	Edebug stops execution: 1. when the next stop point is reached (a stop point are before and after each form inside an instrumented function), 2. when it reaches a breakpoint (which can be set and unset with the following first 3 commands) 3. on a **global break condition**, a conditional expression stored inside the edebug-global-break-expression (using the **X** command below) 4. on an **explicit source breakpoint**: a **`(edebug)`** call inside the source code. Note that breakpoints are ignored in the Go-non-stop mode (started with the **G** command, described above, also set by typing **Q** to stop without breaking.		
Set breakpoint	• `b` • `C-x SPC` • **`C-x X b`**	(**edebug-set-breakpoint** ARG)	Set the breakpoint of nearest sexp. • With prefix argument, make it a temporary breakpoint (it's turned off the first time it stops execution). • This can be done at any time when Edebug is active
Unset breakpoint	• `u` • `C-c C-d` • **`C-x X u`**	(**edebug-unset-breakpoint**)	Clear the breakpoint of nearest sexp.
Set conditional breakpoint	• `x` • **`C-x X x`**	(**edebug-set-conditional-breakpoint** ARG CONDITION)	Set a conditional breakpoint at nearest sexp. • Emacs prompts for a condition. • The condition is evaluated in the outside context. • With prefix argument, make it a temporary breakpoint (it's turned off the first time it stops execution).
Move point to next breakpoint in current definition	`B`	(**edebug-next-breakpoint**)	Move point to the next breakpoint, or first if none past point.

Set global break condition	• `X`
	• **`C-x X X`**
		(**edebug-set-global-break-condition** EXPRESSION)	Set 'edebug-global-break-condition' to EXPRESSION.
			• The expression is tested at every stop point:
			  • if the result is non-nil, then break. Errors are ignored.
			  • This slows down execution, so if not needed set it to nil (the default).

Edebug Views	The following EDebug commands can be used to view aspects of the Emacs buffer and windows status as they were before entry to EDebug. These are is is useful when the code being debugged controls windows and buffers.		
View where am I	• `w` • `C-c C-l` • **`C-x C-a C-l`** • **`C-x X w`**	(**edebug-where**)	Show the debug windows and where we stopped in the program. ☛ This command is also used in the context of the Edebug Evaluation List buffer (see below) with the same behaviour.
Bounce to current point	`p`	(**edebug-bounce-point** ARG)	Bounce the point in the outside current buffer. • If prefix argument ARG is supplied, sit for that many seconds before returning. The default is one second.
View outside window	• `P` • `v`	(**edebug-view-outside**)	Change to the outside window configuration. • Use 'edebug-where' to return.
Toggle save windows	• `W` • **`C-x X W`**	(**edebug-toggle-save-windows** ARG)	Toggle the saving and restoring of windows. • With prefix, toggle for just the selected window. • Otherwise, toggle for all windows.
Evaluation in Edebug	When Emacs is in Edebug mode you can use the following commands to evaluate expression within the "***outside context***", the context of the program being debugged, as opposed to the context of EDebug itself (with some limitations — see the link). For instance when you evaluate an expression, you would not want it to be affected by the operations you performed during EDebug mode (liek the commands you issued). So EDebug saves some and restores the environment of the "program under test" when you evaluate an expression with the following commands.		
Eval Expression	`e`	(**edebug-eval-expression** EXPR)	Evaluate an expression in the outside context. • If interactive, prompt for the expression. • Print result in minibuffer.
Eval Last S-exp	`C-x C-e`	(**edebug-eval-last-sexp**)	Evaluate sexp before point in the outside context. • Print value in minibuffer.
Evaluate Expression in mini-buffer	`M-:`	(**eval-expression** EXP &optional INSERT-VALUE NO-TRUNCATE CHAR-PRINT-LIMIT)	Read a single Emacs Lisp expression in the mini buffer, evaluate it, and print the value in the echo area. • During EDebug session, this is done in the outside context.

Description	Keystroke	Function	Note

EDebug Evaluation List Buffer — evaluation watcher	When in edebug-mode you can use the **E** command to open a *edebug* buffer window where you can evaluate expression interactively within the "***outside context***" with the C-j and C-x C-e command just as you can in the *scratch* buffer. The only difference is that these are are EDebug specialized commands and they use EDebug "***outside context***".
	• When debugging you may want to *watch* the value of some variables or expressions. Write these expressions inside the *edebug* buffer, in groups of 3 lines using the following layout but by creating them by writing the expression in the first line, evaluating it with C-j and then completing it with **C-c C-u**. You can repeat the operation several times with different expressions. The *edebug* buffer should contain 1 or several groups of 3 lines:
	  • line 1: the expression under scrutiny
	  • line 2: its value (you may use **C-j** the first time around to get the value
	  • line 3: a Lisp comment (you may want to insert it yourself if the value is several lines. No need to add dashes (**C-c C-u** will do it).
	• Once this is setup, return to the "program under test" with C-c C-w and continue the debugging (or tracing). You can the watch the expression changing values as execution of the "program under test" unfolds!		

Visit Eval List buffer	**E**	(**edebug-visit-eval-list**)	Switch to the evaluation list buffer "*edebug*".
Evaluate expression before point & insert value	**C-j**	(**edebug-eval-print-last-sexp**)	Evaluate sexp before point in outside environment; insert value. • This prints the value into current buffer.
Evaluate expression before point and print value in mini buffer	**C-x C-e**	(**edebug-eval-last-sexp**)	Evaluate sexp before point in the outside environment. • Print value in minibuffer.
Update the value of a watch group	**C-c C-u**	(**edebug-update-eval-list**)	Replace the evaluation list with the sexps now in the eval buffer.
Delete a watch group	**C-c C-d**	(**edebug-delete-eval-item**)	Delete the item under point and redisplay.
Return to the debugger	**C-c C-w**	(**edebug-where**)	Return to the the debug windows, where we stopped in the program.
Edebug Trace Buffer	By default during debugging nothing is stored in the trace buffer. To log execution of the stop points during debugging in the *debug-trace* buffer, set the *debug-trace* variable to non-nil. You can also use edebug-trace function in your code to trace information during execution of code even if Edebug is not active.		
Explicit call to trace		(***edebug-trace*** *FMT &rest ARGS*)	Convenience call to 'edebug-trace-display' using 'edebug-trace-buffer'. ☛ This is **not** an Emacs command; it's a **function** you can use in your code to force an explicit trace log.
EDebug Coverage Testing Support	Edebug provides rudimentary coverage testing and display of execution frequency. Each form is considered covered if it has returned two different values since the beginning of testing. This must be enabled by setting the ***edebug-test-coverage*** variable to non-nil. At the end use the **C-x X =** to put coverage comments inside source code (use one undo to remove it all).		
Display Freq Count	**C-x X =**	(**edebug-display-freq-count**)	Display the frequency count data for each line of the current definition. • The frequency counts are inserted as comment lines after each line, and you can undo all insertions with one 'undo' command. • The counts are inserted starting under the '(' before an expression or the ')' after an expression, or on the last char of a symbol. The counts are only displayed when they differ from previous counts on the same line. • If coverage is being tested, whenever all known results of an expression are 'eq', the char '=' will be appended after the count for that expression. Note that this is always the case for an expression only evaluated once. • To clear the frequency count and coverage data for a definition, reinstrument it.
Other Edebug commands	The following commands are available stop EDebug or view results that were printed in the minibuffer.		
Abort	• **a** • **C-]** • **C-x X a**	(**abort-recursive-edit**)	Abort the command that requested this **recursive edit** or minibuffer input.
Quit to top level	• **q** • **C-x X q**	(**top-level**)	Exit all recursive editing levels. However, instrumented code protected with ***unwind-protect*** or ***condition-case*** forms may resume debugging. • This also exits all active minibuffers.
Quit Nonstop	• **Q** • **C-x X Q**	(**edebug-top-level-nonstop**)	Set mode to Go-nonstop, and exit to top-level: don't stop even for protected code. • This is useful for exiting even if 'unwind-protect' code may be executed.
Previous result	**r**	(**edebug-previous-result**)	Print the previous result.
Show Backtrace	**d**	(**edebug-backtrace**)	Display a backtrace that is just a list of function calls. This is not a complete backtrace like you get with the debug system. But, as documented it is "Better than nothing…"

Description	Keystroke	Function	Note
Profiler	Emacs has a built-in profiler that can be started with the command below and a command to stop it and get a report. No instrumentation is required to use this standard profiler. **Workflow:** 1. Start profiler with: `M-x profiler-start` 2. Execute code that must be profiled 3. Open the report with: `M-x profiler-report` 4. Stop the profiler with: `M-x stop-profiler` 5. To reset all data before profiling again: `M-x profiler-reset`		
Start the profiler		(**profiler-start** MODE)	Start/restart profilers. • MODE can be one of 'cpu', 'mem', or 'cpu+mem'. • If MODE is 'cpu' or 'cpu+mem', time-based profiler will be started. • Also, if MODE is 'mem' or 'cpu+mem', then memory profiler will be started.
Open profiler report.		(**profiler-report**)	Report profiling results. The report is opened in a **XX-Profiler-Report Date Time** buffer where the XX corresponds to the mode selected when the profiler was started, and the Data and Time correspond to the date/time of the report. The report looks like an outline tree with values and percentage to help identify what consumes the most.
Stop the profiler		(**profiler-stop**)	Stop started profilers. Profiler logs will be kept.
Reset the profiler		(**profiler-reset**)	Reset profiler logs.
Open profile file		(**profiler-find-profile** FILENAME)	Open profile FILENAME.

ELProfiler	A separate profiler was written by Barry Warsaw: elp. The ELP package provides several functions to instrument code for profiling. This profiler is much more flexible but code must be instrumented and you must identify what functions to profile (with the elp-instrument- functions). You can also identify a "master" function: the profiler will only capture data during the execution of that function. There can be only one master function.
	To use the profiler, select the functions to instrument by using one of the tree `elp-instrument-` functions. This profiler allows you to concentrate on specific functions and ignore the remainder of Emacs.
	ELProfiler customization user option variables:
	• **elp-reset-after-results**: controls whether information is reset after display:
	  • Non-nil means reset all profiling info after results are displayed.
	  • Results are displayed with the 'elp-results' command.
	• **elp-use-standard-output**: control profiler output:
	  • If non-nil, output to 'standard-output' instead of a buffer.
	• **elp-sort-by-function**: control report ordering:
	  • Non-nil specifies ELP results sorting function. These functions are currently available:
	  • 'elp-sort-by-call-count' -- sort by the highest call count
	  • 'elp-sort-by-total-time' -- sort by the highest total time
	  • 'elp-sort-by-average-time' -- sort by the highest average times
	  • You can write your own sort function. It should adhere to the interface specified by the PREDICATE argument for 'sort'. Each "element of LIST" is really a 4-element vector where:
	  • element 0 is the call count,
	  • element 1 is the total time spent in the function,
	  • element 2 is the average time spent in the function,
	  • and element 3 is the symbol's name string.		

Instrument all functions in a package		(**elp-instrument-package** PREFIX)	Instrument for profiling, all functions which start with PREFIX. • For example, to instrument all ELP functions, do the following: `M-x elp-instrument-package RET elp- RET`
Instrument a function		(**elp-instrument-function** FUNSYM)	Instrument FUNSYM for profiling. • FUNSYM must be a symbol of a defined function.
Instrument a set of functions provided in a list		(**elp-instrument-list** &optional LIST)	Instrument, for profiling, all functions in 'elp-function-list'. • Use optional LIST if provided instead. • If called interactively, prompt for LIST in the minibuffer; type "nil" to use 'elp-function-list'.
Set the profile master function		(**elp-set-master** FUNSYM)	Set the master function for profiling. • This is not required, but if done it forces the profiler to only gather profiling data for the functions called during the execution of that master function. Useful when there's a need to profile the execution of a given function tree under a specific condition.
Stop using a master function		(**elp-unset-master)**	Unset the master function.
Remove the instrumentation in all instrumented functions		(**elp-restore-all)**	Restore the original definitions of all functions being profiled.
Remove instrumentation in a function		(**elp-restore-function** FUNSYM)	Restore an instrumented function to its original definition. • Argument FUNSYM is the symbol of a defined function.
Remove instrumentation in a set of functions provided in a list		(**elp-restore-list** &optional LIST)	Restore the original definitions for all functions in 'elp-function-list'. • Use optional LIST if provided instead.
After profiling, display the results		(**elp-results)**	Display current profiling results. • If 'elp-reset-after-results' is non-nil, then current profiling information for all instrumented functions is reset after results are displayed.
Reset profiling information for all instrumented functions		(**elp-reset-all**)	Reset the profiling information for all functions being profiled.
Reset profiling information for specific function		(**elp-reset-function** FUNSYM)	Reset the profiling information for FUNSYM.
Reset profiling information for the list of specified functions		(**elp-reset-list** &optional LIST)	Reset the profiling information for all functions in 'elp-function-list'. • Use optional LIST if provided instead.
ESUP - Emacs Start Up Profiler	The ESUP package is a specialized profiler: it profiles Emacs startup only: code called from the init.el file. Very useful to find what is slowing down Emacs on startup. ESUP profiles Emacs startup time by launching a new Emacs process from Emacs and examining all code executed at startup. Requires the **esup** external package. PEL activates it when the **pel-use-esup** customization variable is set to **t**. To use: open Emacs in graphics mode. Type: `M-x esup` (with PEL you can type `<f11> ? e P`). Wait for an *esup* buffer to open with the results.		
Profile Emacs startup code	`<f11> ? e P`	(**esup** &optional INIT-FILE &rest ARGS)	Profile the startup time of Emacs in the background. • If INIT-FILE is non-nil, profile that instead of USER-INIT-FILE. • ARGS is a list of extra command line arguments to pass to Emacs.
	The esup profiler has several limitations: 1) it only supports Emacs running in graphics mode. 2) esup steps into `require' and `load' forms at the top level of a file but not if they are enclosed in any other statements. This limits its usefulness when conditional loading is located in the init.el file and when the use-package macros are used. Both of these techniques are used by PEL to reduce init time.		